Not All Features Are Created Equal: A Mechanistic Study of Vision-Language-Action Models

Bryce Grant* **Xijia Zhao*** **Peng Wang†**
Case Western Reserve University
{bag100, xxz1277, pxw206}@case.edu
https://cwru-aism.github.io/vla-interp-page/

Abstract

A fine-tuned VLA policy places the alphabet soup into the basket on command, if given one, just as was demonstrated during fine-tuning. However, when we move the basket five centimeters to the left, it places the soup on the floor off the table. To understand what the policy is thinking, we developed an interpretability pipeline. Guiding the policy with task A's activations by injecting them into the action expert as the policy observes scene B causes $\pi_{0.5}$ to reproduce task A's motor trajectory during 99.6% of episodes (n=1,968). However, this override effect is not universal and it diminishes to 52.1% on SmolVLA where the interleaved fusion attenuates it, and to 57.0% on GR00T N1.5. An injected episode lacks understanding and awareness of the visible scene and instead, it is bound directly to absolute workspace coordinates. This provides the mechanistic evidence explaining the brittleness of VLAs to perturbations that concurrent benchmarks observe. Binding scene to action also holds when deploying to a physical UR10e arm: directed replay caused the arm to replan to the commanded task instead of the instructed one during 26/26 rollouts in one orientation and 18/26 rollouts in another, compared to failing every time in the no-injection and random-direction baselines. Once we physically move the source object used during recording 10–15 cm away from its expected position, the arm grasps once, then releases at that recorded position in 50/50 of rollouts, picking up the wrong block if another happens to be there or closing its hand at empty air if none does. Systematically applying these interventions to five VLAs ($\pi_{0.5}$, OpenVLA-OFT, X- VLA, SmolVLA, GR00T N1.5), and ACT as a language-free baseline, across LIBERO, MetaWorld, SimplerEnv, and ALOHA environments and over 420,000+ rollouts yields three results: all architectures encode the provided instruction, and four of the five architectures weight it subordinately to vision whenever the goal can be discerned from the scene alone; given standard replacement write-back the specific SAE pooling operation that allows for preserved behavior is specific to architecture ($\pi_{0.5}$ uses a per-token pooling while SmolVLA uses one that is indifferent); and finally goal selection and motor generation decomposes into VLM and expert streams wherever the two are made available. We have developed a platform to allow for reproducibility and analysis of our results across all of our trained SAEs, concepts, and rollouts: **Action Atlas** (https://action-atlas.com). Code: https://github.com/CWRU-AISM/action-atlas.

*Equal contribution.
†Corresponding author: pxw206@case.edu

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

1 Introduction

Vision-Language-Action (VLA) policies [Zitkovich et al., 2023, Kim et al., 2024, Black et al., 2025, Physical Intelligence, 2025, Physical Intelligence et al., 2025] fine-tuned on LIBERO [Liu et al., 2023], a simulation benchmark for robots, achieve over 90% success rates (Tab. 2). Perturb the object's position by a few centimeters and the policy fails [Zhou et al., 2025, Fei et al., 2025] while the instruction stays the same and the object is visible. Are VLAs following instructions and picking out the correct object? Or are they replaying a motor program bound to where the object sat in their training distribution? If a VLA-driven robot fails, there is no principled way to understand why other than observing its behavior, whereas in traditional robotics the kinematics and control models can be inspected by construction [Craig, 2018].

Sparse Autoencoders (SAEs) allow us to extract sparse monosemantic features from the polysemantic activations of language models [Cunningham et al., 2024, Bricken et al., 2023, Templeton et al., 2024]. They have been used to find safety-relevant features at scale [Templeton et al., 2024] and can be used to steer activations without retraining [Turner et al., 2023, Rimsky et al., 2024]. There have been multiple concurrent works that have applied these techniques to VLAs (Tab. 1). Häon et al. [2025] and Buurmeijer et al. [2026] steer scalar values like speed and height. Swann et al. [2026] and Jin et al. [2026] learn SAEs on a single policy and steer individual features. Miao et al. [2026] raises success along a learned subspace. All of these works edit one direction on one model with the task held fixed. None of them address the question of whether the policy is acting on the instruction and object or if it is replaying a trajectory toward a location. Steering one feature will not allow us to distinguish between these two, because the object and its training-time location co-vary in every learned feature.

We replace the computation wholesale to see if we can distinguish between these two (Fig. 1). We collect activations from a policy while it is doing task A in scene A, then we inject those activations into the policy while it is looking at scene B and told to do task B. If the policy has a concept of the task it should be able to use scene B and move to object B. However, if it is replaying coordinates it will move along its path from task A. We test this across 5 VLAs ($\pi_{0.5}$ [Physical Intelligence, 2025], OpenVLA-OFT [Kim et al., 2025], X-VLA [Zheng et al., 2026], SmolVLA [Shukor et al., 2025], GR00T N1.5 [Bjorck et al., 2025]) and 1 language-free control (ACT [Zhao et al., 2023]) on 4 different benchmarks with over 420,000 rollouts. The models range from 80M to 7B parameters and use flow matching, diffusion, continuous regression and a conditional variational autoencoder (CVAE) to generate actions. When injecting activations from another task, $\pi_{0.5}$ moves along the path of the injected task 99.6% of the time (1,968 episodes) while looking at the destination scene (Fig. 2). We show the rate per model and label which metric was used to calculate it (ties included) in Tab. 10. We denote this as a *coordinate-bound motor program*, an activation-level computation that will replay the path of the injected task, anchored to workspace coordinates, when given a new scene. This gives a mechanistic account of the brittleness reported by Zhou et al. [2025].

Three more patterns hold across the models we test. (i) There is task-dependent language sensitivity: prompts will affect less behavior if the vision modality knows what the task is, however we find they are easily decodable from the residual stream via linear probing (§4.4).

(ii) Specialization in the pathway between goals being selected within the VLM pathways and actions generated by experts when distinct. This is demonstrated on SmolVLA and similar behavior can be seen on $\pi_{0.5}$ and GR00T. As well as SmolVLA being resistant to cross-task overriding when testing on LIBERO and MetaWorld due to its interleave of fusion layers (§4.5).

(iii) Whether or not SAEs benefit from mean pooling over their features varies by architecture. We find that with our default replace method of writing the SAE reconstruction back into the model $\pi_{0.5}$ and OFT need to maintain per token information and SmolVLA does not care. X-VLA and the GR00T Diffusion Transformer (DiT) allow for mean pooling as long as the reconstruction is written back as a residual. When we add an arm to contrast temporally adjacent codes we see degradation on all models. (§4.6).

Our contributions include:

- An analysis of multiple architectures using mechanistic interpretability techniques on four different benchmarks. Each finding is specific to each model tested.

- Show that there is a mechanism behind the models creating coordinates that align with a specific task. We use both cross task (§4.3) and pathway specialization (§4.5) to find this and show that it is the cause for Zhou et al. [2025]'s findings of brittleness. We also replicate this behavior on a physical UR10e robot (§4.7).

- Findings on using SAEs on VLA models. We test three arms of SAEs (per-token, mean pool and temporal contrastive) and show that different models like different types of SAEs. With the temporal arm we see adjacent code cos similarity rise to 1 on all models ($\cos_{\text{adj}}$:$0.86 \rightarrow 0.997$).

- Creation of Action Atlas (https://action-atlas.com). An open source tool to explore all 520+ SAEs we trained and their 79 concepts we found and over 10,000 videos.

2 Related Work

RT-2 [Zitkovich et al., 2023] established VLAs by using the capabilities of VLMs to output actions in a discrete format that robots can follow. OpenVLA-OFT improved on this by outputting continuous actions and reached 97.1% success on LIBERO [Kim et al., 2025]. There has been further research that has removed auto-regressive decoding of actions. Such as $\pi_{0.5}$ [Physical Intelligence, 2025] which uses flow matching to predict actions with a separate action expert. These methods avoid errors from binning of actions into discrete values. The set of models we study covers many of these variations from 80M ACT to 7B OFT (see Table 6). Others have found similar results to us when evaluating robustness of these models [Fei et al., 2025, Zhou et al., 2025]. Moving objects slightly caused OpenVLA and π_0 to fail at 0.00 success on all LIBERO tasks and $\pi_{0.5}$ to get 0.08–0.38 success (Original scores ranged from 0.92 to 0.98) [Zhou et al., 2025]. Changing the instructions given to the model dropped success to 0.00 on most tasks. Even when the instructions still conveyed the correct meaning. Which leads us to believe the models may be learning the instruction but not using it to help solve the task.

We found that SAEs made for language models need to be modified to work on VLAs. Per-token preservation matters because action tokens carry phase structure that text does not have (§4.6). Some VLAs use discrete tokens to output actions, which breaks SAE insertion (App. C.1).

Concurrent work on VLA interpretability studies one architecture with one intervention. Häon et al. [2025] steer feed-forward directions of π_0/OpenVLA to change behaviors such as speed and height on a real-world UR5. Swann et al. [2026] learn SAEs on $\pi_{0.5}$ and OpenVLA and steer features, one of which selects an object when given an empty prompt. They do this on DROID hardware. Buurmeijer et al. [2026] also steer kinematics and object choice on $\pi_{0.5}$ using a linear observer-controller on DROID hardware. Miao et al. [2026] gate the action expert toward learned success subspaces. They are able to increase success rate but do not change what task the model is trying to complete. Jin et al. [2026] learn event-grounded SAEs and find that they cannot semantically steer the feed-forward layers to select an object. Khan et al. [2025] and Molinari et al. [2025] use SAEs and world-model probes on a single policy. All of these works edit one direction on one model with the task held fixed. We extend this work by injecting activations from another task, comparing SAE pooling methods, separating pathways and replicating on hardware (Tab. 1). See App. A for more related work.

Table 1: Our study and concurrent VLA-interpretability work. Arch.: architectures intervened on. FFN: feed-forward network. X-task: cross-task activation injection. Pool/Path: SAE pooling comparison and VLM-expert pathway split. HW: real robot.

Work	Arch.	Intervention	X-task	Pool/Path	HW
Häon et al. [2025]	2	FFN, kinematic	no	no/no	UR5
Swann et al. [2026]	2	SAE feature	no	no/no	DROID
Buurmeijer et al. [2026]	1	linear, kinematic + object	no	no/no	DROID
Miao et al. [2026]	3	subspace, success	no	no/no	DROID
Jin et al. [2026]	2	SAE, suppression	no	no/no	ALOHA
Khan et al. [2025]	1	SAE direction	no	no/no	sim
This work	**6**	**injection + SAE**	**yes**	**yes/yes**	**sim + UR10e**

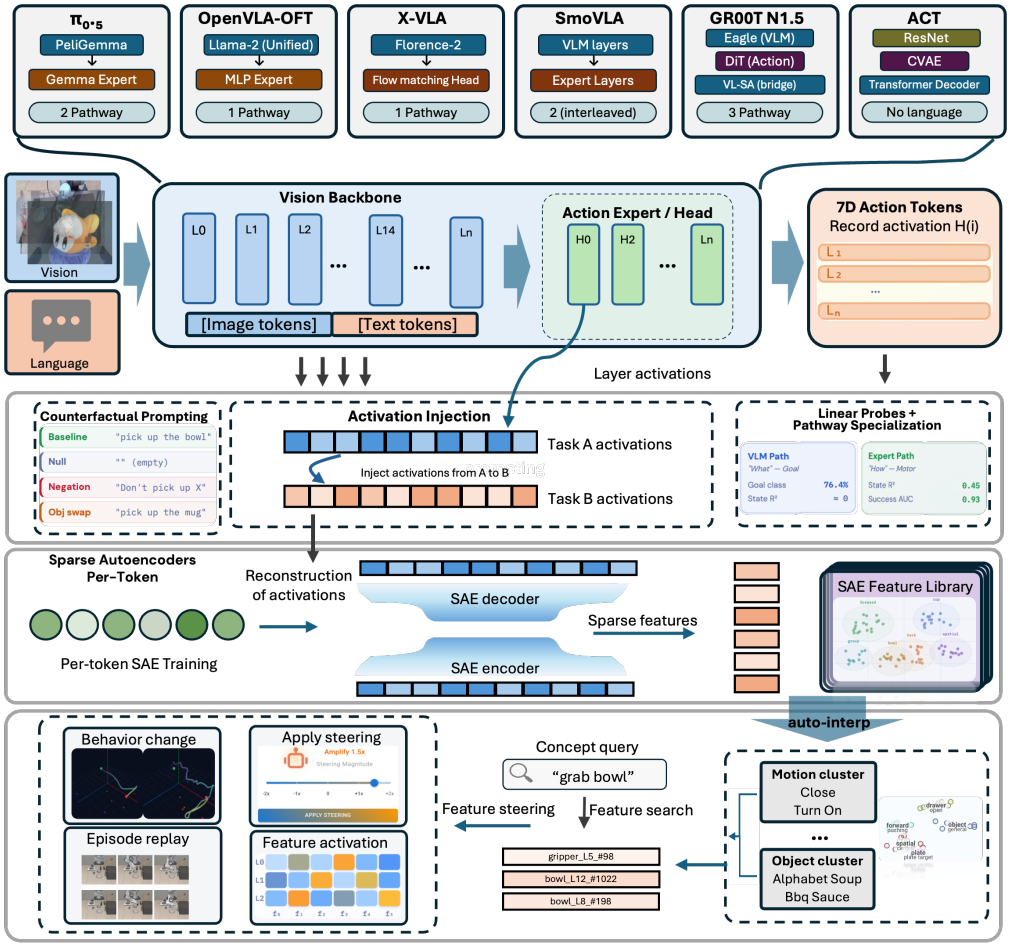


Figure 1: **Methodology overview.** Top: record a VLA's activations during a successful rollout and replay them under cross-task injection and counterfactual prompts. Middle: per-token SAEs decompose the same activations into features. Bottom: features are clustered, searched, and validated by ablation and steering in Action Atlas.

3 Method

Our pipeline is closed-loop in that we use rollouts as our ground truth for any claims about mechanism (Fig. 1). We can intervene on the policy using activation injection and counterfactual prompts. Both allow us to extract causality from the resulting change in behavior. Activation injection allows us to replay computations within the forward pass and counterfactual prompts allow us to modify the language input while keeping the scene constant. We can then use SAEs to extract features from these saved activations and localize causality to certain layers and subspaces. Linear probes can be used to understand what information the residual stream contains at each layer [Li et al., 2026a]. We validate with the same intervention techniques. Any feature we find with our SAEs counts as a mechanism only if it causes the same effect through injection and rollout.

3.1 VLA Architectures Under Study

We use five VLAs and one model without language as a control. These include: $\pi_{0.5}$ [Physical Intelligence, 2025] (3B params, flow matching), OpenVLA-OFT [Kim et al., 2025] (7B params, continuous L1 loss), X-VLA [Zheng et al., 2026] (1B params, soft-prompted flow), SmolVLA [Shukor et al., 2025] (450M params, VLM-expert interleaved), GR00T N1.5 [Bjorck et al., 2025] (3B params, DiT-Eagle-VL-SA) and ACT [Zhao et al., 2023] (80M params, CVAE, no language). This covers

models from 80M to 7B params and four types of action generation: flow matching [Lipman et al., 2023], continuous regression, diffusion and CVAE decoding (see Tab. 6).

3.2 Activation Injection

Activation injection extends activation patching [Meng et al., 2022] to full rollout episodes: we replace activations from one episode with those from another during inference. Given a successful source episode A and target episode B, we record the layer activations $\{\mathbf{H}^{A,(\ell)}\}$ during the rollout of A, then at every timestep t of rollout B, we override the residual stream at the chosen layers,

$$\mathbf{H}_t^{B,(\ell)} \leftarrow \mathbf{H}_t^{A,(\ell)}, \qquad \ell \in \mathcal{L}_{\text{inject}}, \tag{1}$$

and let the rest of the forward pass proceed. The injection set $\mathcal{L}_{\text{inject}}$ ranges from a single layer to all layers. When the prompts of A and B produce residual streams of varying sequence lengths, we overwrite the fixed 50 action-token positions and let the image and prompt tokens pass through without modification. When the target runs longer than the recorded source ($T_B > T_A$), injection ends at $t{=}T_A$ and the target policy continues unmodified (this impacts $<$5% of cross-task episodes; restricting to $T_B \leq T_A$ shifts override $\leq$1pp on $\pi_{0.5}$ `libero_goal`).

We have three injection settings. **Null injection** will use the correct prompt on the source episode and no prompt on the target. **Same-scene injection** will use the same scene for both episodes but instruct them to interact with different objects. **Cross-task injection** will use two different scenes. For models with multiple pathways ($\pi_{0.5}$, SmolVLA, GR00T) we also inject into each pathway individually to isolate their contributions.

3.3 Counterfactual Prompting

We change the text prompt to see how much the language affects behavior. Each episode will have one of six prompt conditions: the correct prompt, null (empty string), negation (e.g., "Don't pick up X"), motor instructions (e.g., "Move slowly"), object swap (replace the target object's name) and temporal switch (change the prompt partway through the episode). We also use counterfactual prompts on SmolVLA for the different difficulty levels on MetaWorld (easy, medium, hard, very hard).

3.4 Sparse Autoencoders for VLAs

We train SAEs on activations related to actions, using TopK sparsity [Gao et al., 2025] with $k{=}64$ and an expansion factor of $m{=}4d$ or $m{=}8d$. It is important to keep token-level information for some models (§4.6). When we take the mean over the 50 action tokens for $\pi_{0.5}$ we lose the ability to distinguish between approaching the object, interacting with it and finishing the task. Rollout success goes from 89.9% to 3.3% (average over all four LIBERO suites and when sweeping over layers for just `libero_object` we see a similar trend, 93.8% to 8.9%), despite a normalized MSE ≤ 0.05 ($R^2 > 0.95$). We report normalized MSE as in Gao et al. [2025]. We process each token individually as the conservative default. More on pooling by model is in §4.6 and App. H.2. To find concept-specific features we use frequency-weighted contrast ($\text{score}_f = d_f \times \text{freq}_f$). For each feature we take the standardized difference in activation between tasks containing the concept and those that do not. This is the Cohen's d (d_f) [Cohen, 1988] of the feature. We then multiply this by freq_f, the fraction of samples in which the feature is in the top-k active features. We train over 520 SAEs across the five VLAs (see Tab. 7 for the number of SAEs per model), finding 79 unique concepts related to manipulation (the union of concept names across models; the per-model lists overlap heavily). These concepts cover different aspects of motion, objects and space.

3.5 Linear Probes for Action Prediction

Linear probes [Alain and Bengio, 2017] test whether action information is linearly decodable from intermediate representations. We train ridge regression probes per action dimension and verify causality by projecting out the probe direction.

3.6 Metrics

We evaluate three primary metrics. **Action Cosine Similarity** measures behavioral alignment between episodes, $\cos(\mathbf{a}^X, \mathbf{a}^Y) = \langle \mathbf{a}^X, \mathbf{a}^Y \rangle / (\|\mathbf{a}^X\| \|\mathbf{a}^Y\|)$, computed over the full per-token action vectors. **Task Success** is a binary indicator determined by the environment's built-in success criteria. We report the change in success rate (SR) as $\Delta\text{SR} = \text{SR}_{\text{intervention}} - \text{SR}_{\text{baseline}}$. **Override Rate** quantifies how often the injected behavior dominates the prompted behavior in cross-task injection. Let $\Delta_t = \cos(\mathbf{a}_t^{B|A\text{-inj}}, \mathbf{a}_t^A) - \cos(\mathbf{a}_t^{B|A\text{-inj}}, \mathbf{a}_t^B)$. We define the override rate conditional on exclusion of a cosine-tie band $|\Delta_t| \leq \tau$,

$$\text{OR} \;=\; \Pr[\,\Delta_t > \tau \mid |\Delta_t| > \tau\,], \tag{2}$$

with $\tau = 0.05$. The share of steps inside the tie band varies by architecture, from 0.3% on $\pi_{0.5}$ to 60.0% on X-VLA, so we also report *tie-inclusive* override, $\Pr[\Delta_t > 0]$ over all injection steps, for every model (Tab. 10, Fig. 2).

4 Experiments

Our results show that the scene determines which motor program is selected (§4.2, §4.3). Language only plays a role when there is ambiguity in the scene (§4.4). Different pathways are responsible for different parts of this process. The VLM selects the coordinate-bound motor program and the expert binds it to coordinates (§4.5). We can use SAEs to find features that represent this information, with a pooling choice that depends on the architecture (§4.6). We test this binding on a real robot arm (§4.7).

4.1 Experimental Setup

We collect 420,000+ rollout episodes across 12 experiment types and up to 50 tasks per environment (scale in Table 7), on 8×A100-80GB, RTX 5090, and 2×RTX 4090, over four benchmarks: **LIBERO** [Liu et al., 2023] (4 suites, 40 tasks), **MetaWorld** [Yu et al., 2020] (50 tasks, 4 difficulty levels), **SimplerEnv** [Li et al., 2024b] (10 tasks, 2 embodiments), and **ALOHA** [Zhao et al., 2023] (2 bimanual tasks).

4.2 Visual Context Dominates Action Generation

Visual context dominates action generation across all six architectures (Tab. 2). By injecting the base activation from the visual pathway when using the null prompt we get cosine similarity back to ≥ 0.99 and get task success rates of 14–80%. If we set one layer to zero we see both X-VLA and GR00T-DiT go to 0%. SmolVLA can survive single expert-layer zeroing (19.2–76.5% across suites). We found greater success rates when injecting into dual-pathway models ($\pi_{0.5}$ had success rates of 70–80% per suite at n=50/suite, against null injection controls of 22%, 6.0% and 62%, Tab. 2). And smaller success rates for single-pathway models such as OFT (14–15%, n=120 per layer). We believe this is due to language being more involved with the visual pathway in these models as they have to rely on language more to understand the prompt (§4.4). We find that the model recovers some percentage of the success rate lost by the prompt. It restores 88% of the lost success on `libero_goal` and 69% on `libero_object`, while the `libero_spatial` contrast is not significant (p=0.13), the suite where an empty prompt costs 26%. Per-model layer-zeroing profiles in App. H.

When we inject activations from prompt B into rollouts of prompt A within the same scene we can steer towards the injected target on every multi-pathway model tested. $\pi_{0.5}$ also tolerates zeroing out later layers, similar to SmolVLA. Once vision has picked an action, what does the activation represent? See §4.3.

4.3 Motor Programs Bind to Absolute Spatial Coordinates

Is the injected activation representing a concept of the task ("pick up the cup") or a coordinate-bound motor program? We test this by injecting activations across tasks. Here we overwrite the expert-pathway residual stream at layers 16 and 17 across all 50 action-token positions at every timestep. For all five language-capable VLAs, destination-task success collapses, and trajectory analysis shows behavior steered toward source-task positions (Tab. 3 with ACT serving as the language-free control).

Table 2: Visual-pathway influence per suite. Baselines come from a unified 25-ep×10-task batch (n=250/suite), and the OFT null prompt is n=50/suite. *One matched $\pi_{0.5}$ batch (n=50/suite) with its own baselines of 90.0%, 92.0% and 88.0% (App. H). **From the n=500 sweep of Tab. 4. Same-scene: baseline activations injected into a same-task episode. Rec.: OFT null prompt plus injected baseline activations. Zero-layer rows average the layer sweep. †The public GR00T spatial checkpoint underperforms the reference (App. K).

Model	Condition	Goal	Object	Spatial	Long / 10
$\pi_{0.5}$	Baseline	93.2%	87.6%	92.8%	86.0%
	Null prompt, no injection*	6.0%	22.0%	62.0%	56.6%**
	Null + PaliGemma-ALL inject*	80.0%	70.0%	78.0%	—
	Same-scene inject ALL	91.0%	87.5%	88.2%	75.2%
OFT	Baseline	98.8%	99.2%	92.4%	91.2%
	Null prompt, no injection	12.0%	96.0%	58.0%	—
	Null + inject zero-baseline (rec.)	14.1%	14.6%	14.4%	13.5%
X-VLA	Baseline	98.8%	98.8%	95.6%	87.6%
	Null prompt, no injection	10.0%	60.0%	48.0%	28.0%
	Zero any single layer	0.0%	0.0%	0.0%	0.0%
SmolVLA	Baseline	74.8%	87.6%	65.6%	41.2%
	Zero expert layer (mean)	71.0%	76.5%	61.8%	19.2%
GR00T N1.5	Baseline	93.6%	95.6%	69.2%†	77.2%
	Null source (inject ALL DiT)	3.3%	3.3%	—†	3.3%
	Zero any DiT layer (L0–L15)	0.0%	0.0%	—†	0.0%
ACT (ALOHA)	Baseline (n=100/task)	100.0% TransferCube, 78.0% Insertion			
	Mask workspace grid (2,2), n=20	10.0% TransferCube, 20.0% Insertion			

Table 3: Cross-task injection across six models. Dst. SR: destination-task success under injection. Override rate: tie-band-conditional at τ=0.05 (Eq. 2), with tie-inclusive rates in Tab. 10. ACT's Dst. SR is its uninjected Insertion rate on the same 10 seeds.

Model	Pairs (n)	Dst. SR	Override rate	Notes
$\pi_{0.5}$	1,968	2.6%	99.6%	goal, spatial, object
OFT	1,079	5.0%	96.4%	90.8–100.0% per suite
X-VLA	3,150	0.0%	99.8%	LIBERO; WidowX 54.2%
SmolVLA	720	0.0%	52.1%	expert 78.6%, VLM 25.6%
GR00T	270	0.0%	57.0%	goal 85.6%, object 52.2%, long 33.3%
ACT (ctrl.)	10	70.0%	0.0%	no-op (cos =1.0)

Analyzing the displacement of the injected trajectories (Figure 2), we find that injection decreases success and still steers behavior. $\pi_{0.5}$, X-VLA, and OFT all show strong source dominance ($\geq$96% conditional on the tie band, and 52.8–99.6% when ties are included). GR00T shows moderate source dominance, with 57.0% under action cosine and 6.7% under end-effector velocity. SmolVLA's expert pathway overrides at 3.07× the rate of its VLM pathway. Objects do move, but they are the objects of the destination task. Using DTW on the end-effector trajectories, we find source dominance of 86–91% on `libero_goal` and `libero_spatial` (App. H.3).

A random direction of the same magnitude at the same layers gives chance-level override (App. C.3), so the effect comes from the injected direction. When we align the source and destination tasks by motion phase, the override rate remains high at 94.4% (App. H.3). Reducing the fine-tuning data by an order of magnitude gives the same result (App. N), as does halving the injection strength (App. L). The robot therefore moves to where the source objects would have been, and its motor program is tied to absolute workspace coordinates, which agrees with prior findings [Fei et al., 2025, Zhou et al., 2025]. We also observe this in our physical experiments (§4.7).

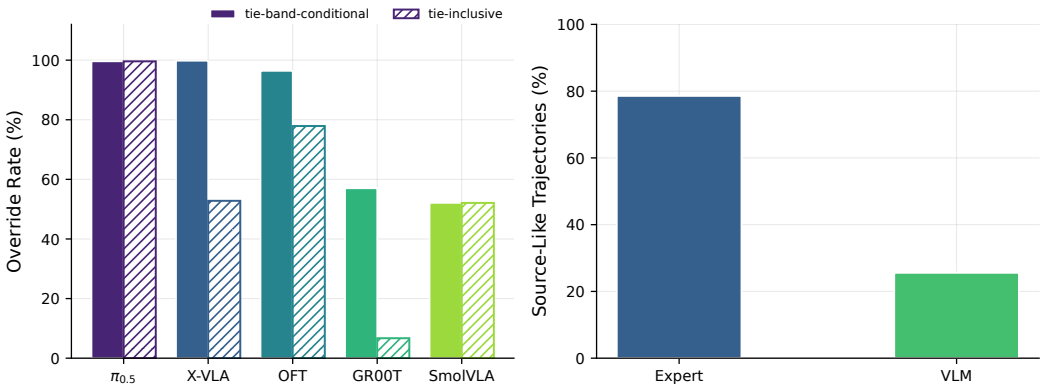


Figure 2: **Cross-task override rates.** Left: tie-band-conditional (solid) and tie-inclusive (hatched) override per model, with metrics and CIs in Tab. 10. Right: SmolVLA override by injected pathway on LIBERO (n=720).

4.4 Language Becomes Necessary Only When Vision is Ambiguous

If the visual context determines the action, then language should only play a role when the scene does not provide enough information to identify the task. We test this with counterfactual prompts across five language-capable VLAs (per-model n in App. H) and find that it holds in graded form. Task structure sets the ordering of prompt tolerance. This ordering is consistent across models, but its magnitude changes between models. Across the four LIBERO suites and four levels of MetaWorld difficulty, every model tested on both benchmarks has a higher success rate under a null or incorrect prompt on tasks with clear visual cues (`libero_object`, MetaWorld easy) than on visually ambiguous tasks (`libero_goal`, MetaWorld hard), by at least a factor of two. $\pi_{0.5}$ has a success rate of 31.6% with a null prompt on `libero_object` and 5.8% on `libero_goal`, compared to baselines of 91.0% and 92.6% respectively (n=500/cell). $\pi_{0.5}$ does not follow the meaning of instructions. When told `do not {task}`, it completes the task 96.7% of the time (n=30, within-run baseline 86.7%).

The magnitude changes between models and spans the entire range on one suite. On `libero_object`, OFT maintains a success rate of 96.0% when given a null prompt (n=50/cell), $\pi_{0.5}$ falls to 31.6%, and SmolVLA is at 0.0% ($\pi_{0.5}$ and SmolVLA n=500/cell, baselines of 100.0%, 91.0% and 89.2% respectively). The SmolVLA checkpoint for LIBERO either succeeds or fails completely. It succeeds in 0/500 episodes with a null prompt and in 0/12 with an empty, a one-token, or a nuisance prompt, compared to 11/12 with the true instruction. The SmolVLA checkpoint for MetaWorld, however, maintains a success rate of 82.0% with a null prompt on the easy difficulty (Tab. 4).

Even though behavior is largely invariant to the prompt, a linear probe on the late layers of each language-capable model classifies the prompt category with over 99% accuracy (Tab. 19). All models encode the prompt and use it only when they cannot determine the goal from the scene. Huang et al. [2026] found similar results on physical robots and attribute them to visual-encoder drift (App. A).

4.5 VLM Pathways Select the Goal and Expert Pathways Generate the Motor Program

The bind/select division of labor predicts an architectural division, with VLM pathways selecting the goal and motor-expert pathways generating the coordinate-bound motor program. The dissociation is quantified on SmolVLA and reproduced directionally on the other two multi-pathway models. Injecting expert pathways causes active misdirection (reaching to wrong locations), and injecting VLM pathways causes passive stalling (timeout) (Fig. 2, right). On SmolVLA, oracle state-prediction probes recover ground-truth state from expert activations at $4.5\times$ the VLM rate (oracle ratio 0.58 vs. 0.13, Tab. 19). The prediction has two halves, the VLM side decoding the goal better and the expert side decoding the state better. SmolVLA shows the state half and $\pi_{0.5}$ the goal half (a modest 14 pp gap): PaliGemma goal classification rises to 76.4% while expert goal classification settles at 62.6% (App. F.1), and GR00T DiT layers are more ablation-sensitive (40–80% drop) than VL-SA layers

Table 4: Counterfactual prompting across models (n per model in App. H). The n=500 $\pi_{0.5}$ and SmolVLA cells are 50 episodes per task from one fixed init state (LeRobot 0.4.4 [Cadene et al., 2024] pins it), paired on layout across conditions. ‡Matched within-experiment control, within 6 pp of the Tab. 2 baselines. §Not run on MetaWorld.

Model / Suite	Correct Prompt‡	Null	Wrong Obj.§
$\pi_{0.5}$ / object	91.0%	31.6%	—
$\pi_{0.5}$ / goal	92.6%	5.8%	—
$\pi_{0.5}$ / 10	86.4%	56.6%	—
$\pi_{0.5}$ / spatial	91.0%	55.0%	—
OFT / object	100.0%	96.0%	—
OFT / goal	98.0%	12.0%	—
OFT / spatial	92.0%	58.0%	—
X-VLA / object	100.0%	60.0%	60.0–90.0%
X-VLA / goal	94.0%	10.0%	4.0–10.0%
X-VLA / spatial	98.0%	48.0%	44.0–58.0%
SmolVLA / object	89.2%	0.0%	—
SmolVLA / MW easy	85.0%	82.0%	—
SmolVLA / MW hard	62.0%	41.0%	—
GR00T / object	93.0%	70.0%	50.0%
GR00T / goal	97.0%	0.0%	0.0%
GR00T / long	83.0%	67.0%	47.0%

(Fig. 11), consistent with the same expert-encodes-state pattern under a different fusion. The goal side is causal. An edit to the VLM pathway at the instruction positions restores goal selection with no instruction on four of five architectures, and matched random or wrong-task edits do not. OFT, which has no separate VLM pathway, does not respond (App. G). Where a VLA has a separate expert, the expert holds the coordinates and the VLM holds the goal and an edit on the VLM side selects it.

4.6 Pooling Preference Is Architecture-Tied and Temporal Contrast Collapses Phase

Reading this substrate with SAEs requires us to decide how we want to pool over action tokens, which can vary by model architecture. We train TopK SAEs on all pathway layers (Eqs. 3–5; 520+ SAEs total) and observe two trends across architectures.

Single-feature ablation has no impact on 28–92% of concept/task combinations (Fig. 7), consistent with concepts being distributed redundantly across multiple features. Certain layers show higher sensitivity, and the action pathway differs from the language stack (Fig. 5). Both the $\pi_{0.5}$ expert (18 layers) and GR00T DiT (16 layers) show that 40% or more of ablated feature/task combinations kill the policy ($\Delta\text{SR} \geq 0.5$) for the majority of their depth, while OFT (32 layers), X-VLA (24 layers) and the SmolVLA expert (32 layers) keep this kill-switch rate under 10% at almost every layer. Model width does not predict feature sensitivity (see model-specific charts in App. H).

We compare the five models with a three-arm rollout setup and observe two behaviors when pooling features (Tab. 12, rollout success rate through the per-token SAE and the mean-pooled SAE). $\pi_{0.5}$ drops from 81% to 3% and OFT from 80% to 7% when using a mean-pooled SAE, indicating they both need to be reconstructed on a per-token basis. X-VLA sees an increase (51% to 80%) and GR00T-DiT performs slightly better (80% to 83%), with SmolVLA staying constant (67.2% and 66.4%). With the temporal-contrastive loss we also observe the collapse of adjacent codes (adjacent-code cosine similarity $\cos_{\text{adj}}$ 0.86 to 0.997) but still have an R^2 of 0.97 or more on $\pi_{0.5}$, X-VLA and GR00T-DiT (SmolVLA 0.940; OFT is data-limited). On all models we tested, this collapse comes with a drop in rollout success. Pooling should be chosen per model and signal type (App. G), and the temporal-contrastive loss should be avoided. All mean-pool values are computed with the replacement write-back. When we use a residual write-back instead, the mean-pool collapse disappears on OFT, GR00T and X-VLA (App. M).

4.7 The Program Replicates on a Physical UR10e

We take the claim from §4.3 out of the simulator and fine-tune $\pi_{0.5}$ on a UR10e. We test 7 tabletop tasks (76.2% base success rate, 160/210, with 30 rollouts per task) and perform the same wholesale expert replay at L16/L17 from plushie-in-bowl to gray-on-blue stacking and vice versa (App. O).

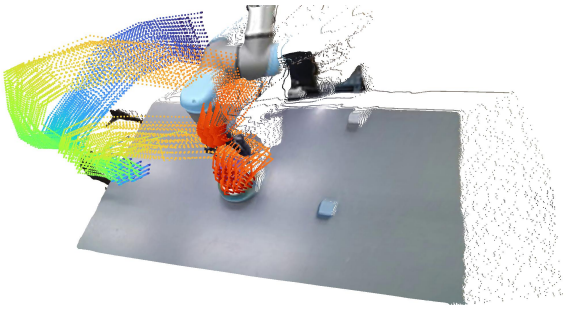


Figure 3: **Directed injection on the UR10e.** The plushie program is injected at expert L16/L17 under the prompt to stack the blocks. The arm reaches the plushie and carries it to the bowl, and the blocks stay untouched. Tracked points run from start (blue) to end (red).

Table 5: Physical cross-task injection on the UR10e, source-classified rate per condition [95% CI].

Condition	Blocks program under plushie prompt	Plushie program under blocks prompt
Directed injection	**26/26** = 100.0% [87.1, 100.0]	**18/26** = 69.2% [50.0, 83.5]
No injection (floor)	0/24 = 0.0% [0.0, 13.8]	0/25 = 0.0% [0.0, 13.3]
Norm-matched random	0/25 = 0.0% [0.0, 13.3]	1/23 = 4.3% [0.8, 21.0]

When injecting our directed activation (Tab. 5) we see the robot perform the source task 26/26 times in one direction and 18/26 in the other. This is compared to 0/24 and 0/25 with no injection on the same scene, and 0/25 and 1/23 for a random direction with the same norm. To separate object tracking from coordinate anchoring, we then displace the source object 10–15 cm from where it was recorded. When we inject the activation the robot goes to the recorded coordinates in 50/50 trials across both directions. The non-injected policy goes to the displaced object (Fig. 3; object tracking done with CoTracker3 [Karaev et al., 2025] and DA3 [Lin et al., 2026]). We also reuse the flow-matching sampling noise from the source task when doing physical replays. This is not matched in our simulation results (App. O).

4.8 Limitations

Simulation carries all cross-architecture results, and hardware covers a single model and task pair (§4.7). Extended limitations and three falsifiable predictions are in App. B.

5 Conclusion

Across six architectures, fine-tuned VLAs learn to represent coordinate-bound motor programs that are selected by the vision pathway and executed by the expert pathway if it exists. When we inject these activations into the action expert we see 99.6% of $\pi_{0.5}$ rollouts follow the source task's trajectory in a new scene. Sensitivity to language depends on the task and the architecture (§4.4). The binding can explain the brittleness seen in benchmarks [Zhou et al., 2025, Fei et al., 2025] and replicates on a real UR10e (§4.7). The goal is represented in the VLM and can be restored with a single edit without instructions on 4/5 architectures. When learning SAEs on VLAs, pooling should be chosen per architecture, and the temporal-contrastive loss hurts all architectures we tested.

Acknowledgments and Disclosure of Funding

This material is based upon work supported by the National Science Foundation Graduate Research Fellowship Program under Grant No. 2437833 and by National Science Foundation Grant No. 2533634. The authors also thank NVIDIA for computing resources provided through the NVIDIA Academic Grant Program.

References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. Do as I can, not as I say: Grounding language in robotic affordances. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 2022.

Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. In *International Conference on Learning Representations (Workshop)*, 2017.

ALOHA 2 Team, Jorge Aldaco, Travis Armstrong, Chelsea Finn, Pete Florence, Jonathan Tompson, and Tony Z. Zhao. ALOHA 2: An enhanced low-cost hardware for bimanual teleoperation. *arXiv preprint arXiv:2405.02292*, 2024.

Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022.

Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

Usha Bhalla, Alex Oesterling, Claudio Mayrink Verdun, Himabindu Lakkaraju, and Flavio P. Calmon. Temporal sparse autoencoders: Leveraging the sequential nature of language for interpretability. In *International Conference on Learning Representations (ICLR)*, 2026.

Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*, 2025.

Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL https://transformer-circuits.pub/2023/monosemantic-features/index.html.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems (RSS)*, 2023.

Hugo Buurmeijer, Carmen Amo Alonso, Aiden Swann, and Marco Pavone. Observing and controlling features in vision-language-action models. In *ICML 2026 Workshop on Mechanistic Interpretability*, 2026.

Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Khalil Meftah, Maxime Ellerbach, Jess Moss, and Thomas Wolf. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. https://github.com/huggingface/lerobot, 2024.

Chilam Cheang, Sijin Chen, Zhongren Cui, Yingdong Hu, Liqun Huang, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Xiao Ma, Hao Niu, Wenxuan Ou, Wanli Peng, Zeyu Ren, Haixin Shi, Jiawen Tian, Hongtao Wu, Xin Xiao, Yuyang Xiao, Jiafeng Xu, and Yichu Yang. GR-3 technical report. *arXiv preprint arXiv:2507.15493*, 2025.

Jiayi Chen, Wenxuan Song, Jiaxin Fang, Ruiqing Yin, Jingbo Wang, Shuai Chen, Jieyuan Pei, Yikai Qin, Feifan Chen, Haodong Yan, Zhide Zhong, Wen Chen, Yan Wang, Yuxiang Gao, and Haoang Li. DFM-VLA: Iterative action refinement for robot manipulation via discrete flow matching. *arXiv preprint arXiv:2603.26320*, 2026.

Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.

Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.

John J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson, Hoboken, NJ, 4 edition, 2018. ISBN 9780133489798.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Yinpei Dai, Jayjun Lee, Nima Fazeli, and Joyce Chai. RACER: Rich language-guided failure recovery policies for imitation learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.

Shaoqi Dong, Chaoyou Fu, Haihan Gao, Yi-Fan Zhang, Chi Yan, Chu Wu, Xiaoyu Liu, Yunhang Shen, Jing Huo, Deqiang Jiang, Haoyu Cao, Yang Gao, Xing Sun, Ran He, and Caifeng Shan. VITA-VLA: Efficiently teaching vision-language models to act via action expert distillation. *arXiv preprint arXiv:2510.09607*, 2025.

Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. PaLM-E: An embodied multimodal language model. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 8469–8488. PMLR, 2023.

Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2025.

Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. LIBERO-Plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.

Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. In *International Conference on Learning Representations (ICLR)*, 2025.

Davide Ghilardi, Federico Belotti, Marco Molinari, Tao Ma, and Matteo Palmonari. Group-SAE: Efficient training of sparse autoencoders for large language models via layer groups. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 18657–18677, 2025.

Catherine Glossop, William Chen, Arjun Bhorkar, Dhruv Shah, and Sergey Levine. CAST: Counterfactual labels improve instruction following in vision-language-action models. *IEEE Robotics and Automation Letters*, 11(10):11785–11792, 2026.

Michal Golovanevsky, William Rudman, Michael A. Lepori, Amir Bar, Ritambhara Singh, and Carsten Eickhoff. Pixels versus priors: Controlling knowledge priors in vision-language models through visual counterfacts. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 24837–24852, 2025.

Bear Häon, Kaylene Stocking, Ian Chuang, and Claire Tomlin. Mechanistic interpretability for steering vision-language-action models. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 2743–2762. PMLR, 2025.

Yutong Hu, Jan-Nico Zaech, Nikolay Nikolov, Yuanqi Yao, Sombit Dey, Giuliano Albanese, Renaud Detry, Luc Van Gool, and Danda Pani Paudel. AR-VLA: Autoregressive action expert for vision-language-action models. In *Robotics: Science and Systems (RSS)*, 2026.

Suning Huang, Jiaqi Shao, Ke Wang, Qianzhong Chen, Jiankai Sun, Yanjiang Guo, Mac Schwager, and Jeannette Bohg. Breaking lock-in: Preserving steerability under low-data VLA post-training. *arXiv preprint arXiv:2604.23121*, 2026.

Yuhua Jiang, Shuang Cheng, Yan Ding, Feifei Gao, and Biqing Qi. AsyncVLA: Asynchronous flow matching for vision–language–action models. *arXiv preprint arXiv:2511.14148*, 2025.

Xinchen Jin, Aditya Chatterjee, Pranav Kumar, and Rohan Paleja. Event-grounded sparse autoencoders for vision-language-action policies. *arXiv preprint arXiv:2605.17204*, 2026.

Sonia Joseph, Praneet Suresh, Ethan Goldfarb, Lorenz Hufe, Yossi Gandelsman, Robert Graham, Danilo Bzdok, Wojciech Samek, and Blake Aaron Richards. Steering CLIP's vision transformer with sparse autoencoders. In *CVPR 2025 Workshop on Mechanistic Interpretability for Vision (MIV)*, 2025.

Nikita Karaev, Yuri Makarov, Jianyuan Wang, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. CoTracker3: Simpler and better point tracking by pseudo-labelling real videos. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6013–6022, 2025.

Chiraag Kaushik, Davis Barch, and Andrea Fanelli. Decomposing multimodal embedding spaces with group-sparse autoencoders. *arXiv preprint arXiv:2601.20028*, 2026.

M. A. Khan, N. Boskov, F. M. Anwar, and M. A. Khan. Controlling vision-language-action policies through sparse latent directions. In *NeurIPS 2025 Workshop on Mechanistic Interpretability*, 2025.

Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 2024.

Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. In *Robotics: Science and Systems (RSS)*, 2025.

Yu Lei, Minghuan Liu, Abhiram Maddukuri, Zhenyu Jiang, and Yuke Zhu. A mechanistic analysis of sim-and-real co-training in generative robot policies. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 65396–65420. PMLR, 2026.

Haoxuan Li, Zhen Wen, Qiqi Jiang, Chenxiao Li, Yuwei Wu, Yuchen Yang, Yiyao Wang, Xiuqi Huang, Minfeng Zhu, and Wei Chen. ConceptViz: A visual analytics approach for exploring concepts in large language models. *IEEE Transactions on Visualization and Computer Graphics*, 32(1):57–67, 2026a.

Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, Dehui Wang, Dingxiang Luo, Yuchen Fan, Youbang Sun, Jia Zeng, Jiangmiao Pang, Shanghang Zhang, Yu Wang, Yao Mu, Bowen Zhou, and Ning Ding. SimpleVLA-RL: Scaling VLA training via reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026b.

Jinming Li, Yichen Zhu, Zhibin Tang, Junjie Wen, Minjie Zhu, Xiaoyu Liu, Chengmeng Li, Ran Cheng, Yaxin Peng, Yan Peng, and Feifei Feng. CoA-VLA: Improving vision-language-action models via visual-text chain-of-affordance. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9759–9769, 2025.

Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, Xiaofan Wang, Bei Liu, Jianlong Fu, Jianmin Bao, Dong Chen, Yuanchun Shi, Jiaolong Yang, and Baining Guo. CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024a.

Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishika Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine, Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and Ted Xiao. Evaluating real-world robot manipulation policies in simulation. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 3705–3728. PMLR, 2024b.

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500, 2023.

Haotong Lin, Sili Chen, Junhao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth Anything 3: Recovering the visual space from any views. In *International Conference on Learning Representations (ICLR)*, 2026.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

Jiaming Liu, Hao Chen, Pengju An, Zhuoyang Liu, Renrui Zhang, Chenyang Gu, Xiaoqi Li, Ziyu Guo, Sixiang Chen, Mengzhen Liu, Chengkai Hou, Mengdi Zhao, KC alex Zhou, Pheng-Ann Heng, and Shanghang Zhang. HybridVLA: Collaborative diffusion and autoregression in a unified vision-language-action model. In *International Conference on Learning Representations (ICLR)*, 2026.

Zhenyang Liu, Yongchong Gu, Sixiao Zheng, Yanwei Fu, Xiangyang Xue, and Yu-Gang Jiang. TriVLA: A triple-system-based unified vision-language-action model with episodic world modeling for general robot control. *arXiv preprint arXiv:2507.01424*, 2025.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Miranda Muqing Miao, Subin Kim, Brandon Yang, and Lyle Ungar. Contrastive conceptor activation steering (COAST): Unlocking vision-language-action models through hidden states. In *ICML 2026 Workshop on Mechanistic Interpretability*, 2026.

Marco Molinari, Leonardo Nevali, Saharsha Navani, and Omar G. Younis. Emergent world representations in OpenVLA. *arXiv preprint arXiv:2509.24559*, 2025.

Aaron Mueller. Missed causes and ambiguous effects: Counterfactuals pose challenges for interpreting neural networks. In *ICML 2024 Workshop on Mechanistic Interpretability*, 2024.

Ali Nasiri-Sarvi, Hassan Rivaz, and Mahdi S. Hosseini. SPARC: Concept-aligned sparse autoencoders for cross-model and cross-modal interpretability. *Transactions on Machine Learning Research*, 2026.

Bruno A. Olshausen and David J. Field. Sparse coding with an overcomplete basis set: A strategy employed by V1? *Vision Research*, 37(23):3311–3325, 1997.

Mateusz Pach, Shyamgopal Karthik, Quentin Bouniot, Serge Belongie, and Zeynep Akata. Sparse autoencoders learn monosemantic features in vision-language models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.

Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. FAST: Efficient action tokenization for vision-language-action models. In *Robotics: Science and Systems (RSS)*, 2025.

Physical Intelligence. $\pi_{0.5}$: A vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 17–40. PMLR, 2025.

Physical Intelligence. $\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities. *arXiv preprint arXiv:2604.15483*, 2026.

Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Chelsea Finn, Sergey Levine, et al. $\pi^*_{0.6}$: a VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.

Joris Postmus and Steven Abreu. Steering large language models using conceptors: Improving addition-based activation engineering. In *NeurIPS 2024 Workshop on Foundation Model Interventions (MINT)*, 2024.

Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, JiaYuan Gu, Bin Zhao, Dong Wang, and Xuelong Li. SpatialVLA: Exploring spatial representations for visual-language-action models. In *Robotics: Science and Systems (RSS)*, 2025.

Moritz Reuss, Hongyi Zhou, Marcel Rühle, Ömer Erdinç Yağmurlu, Fabian Otto, and Rudolf Lioutikov. FLOWER: Democratizing generalist robot policies with efficient vision-language-flow models. In *Proceedings of The 9th Conference on Robot Learning (CoRL)*, volume 305 of *Proceedings of Machine Learning Research*, pages 3736–3761. PMLR, 2025.

Nina Rimsky, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering Llama 2 via contrastive activation addition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15504–15522, Bangkok, Thailand, 2024. Association for Computational Linguistics.

Stéphane Ross, Geoffrey J Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 627–635. PMLR, 2011.

Som Sagar, Jiafei Duan, Sreevishakh Vasudevan, Yifan Zhou, Heni Ben Amor, Dieter Fox, and Ransalu Senanayake. RoboMD: Uncovering robot vulnerabilities through semantic potential fields. In *International Conference on Learning Representations (ICLR)*, 2026.

Haoyuan Shi, Xiancong Ren, Yingji Zhang, Qinfan Zhang, Jiayu Hu, Haozhe Shan, Han Dong, Jinpeng Lu, Yinda Chen, Yi Zhang, Yong Dai, and Xiaozhu Ju. VLA-Trace: Diagnosing vision-language-action models through representation and behavior tracing. *arXiv preprint arXiv:2605.30117*, 2026.

Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

Anushka Sivakumar, Andrew Zhang, Zaber Ibn Abdul Hakim, and Chris Thomas. SteerVLM: Robust model control through lightweight activation steering for vision language models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 23640–23665, 2025.

Samuel Soo, Guang Chen, Wesley Teng, Chandrasekaran Balaganesh, Guoxian Tan, and Ming Yan. Interpretable steering of large language models with feature guided activation additions. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*, 2025.

Samuel Stevens, Wei-Lun Chao, Tanya Berger-Wolf, and Yu Su. Interpretable and testable vision features via sparse autoencoders. *arXiv preprint arXiv:2502.06755*, 2025.

Aiden Swann, Lachlain McGranahan, Hugo Buurmeijer, Monroe Kennedy III, and Mac Schwager. Sparse autoencoders reveal interpretable and steerable features in VLA models. In *Conference on Robot Learning (CoRL)*, 2026.

Shuhan Tan, Kairan Dou, Yue Zhao, and Philipp Krähenbühl. Interactive post-training for vision-language-action models. In *CVPR 2025 Workshop on Foundation Models Meet Embodied Agents*, 2025.

Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C Daniel Freeman, Theodore R Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024. URL https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html.

Harrish Thasarathan, Julian Forsyth, Thomas Fel, Matthew Kowal, and Konstantinos G. Derpanis. Universal sparse autoencoders: Interpretable cross-model concept alignment. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 59304–59325. PMLR, 2025.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Activation addition: Steering language models without optimization. *arXiv preprint arXiv:2308.10248*, 2023.

Junjie Wen, Minjie Zhu, Jiaming Liu, Zhiyuan Liu, Yicun Yang, Linfeng Zhang, Shanghang Zhang, Yichen Zhu, and Yi Xu. dVLA: Diffusion vision-language-action model with multimodal chain-of-thought. *arXiv preprint arXiv:2509.25681*, 2025.

Sihao Wu, Gaojie Jin, Wei Huang, Jianhong Wang, and Xiaowei Huang. Activation steering meets preference optimization: Defense against jailbreaks in vision language model. In *Proceedings of the 17th Asian Conference on Machine Learning*, volume 304 of *Proceedings of Machine Learning Research*, pages 17–32. PMLR, 2025.

Theodor Wulff, Sergio Lanza, Tamara Bila, Angelo Cangelosi, Stefan Wermter, and Igor Farkas. Latent cluster analysis for vision-language-action models. *arXiv preprint arXiv:2609.02634*, 2026.

Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2020.

Yifu Yuan, Haiqin Cui, Yaoting Huang, Yibin Chen, Fei Ni, Zibin Dong, Pengyi Li, Yan Zheng, Hongyao Tang, and Jianye Hao. Embodied-R1: Reinforced embodied reasoning for general robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2026.

Vladimir Zaigrajew, Hubert Baniecki, and Przemyslaw Biecek. Interpreting CLIP with hierarchical sparse autoencoders. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 73918–73956. PMLR, 2025.

Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024.

Jianke Zhang, Xiaoyu Chen, Qiuyue Wang, Mingsheng Li, Yanjiang Guo, Yucheng Hu, Jiajun Zhang, Shuai Bai, Junyang Lin, and Jianyu Chen. VLM4VLA: Revisiting vision-language-models in vision-language-action models. In *International Conference on Learning Representations (ICLR)*, 2026.

Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Ming-Yu Liu, Donglai Xiang, Gordon Wetzstein, and Tsung-Yi Lin. CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023.

Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, Ya-Qin Zhang, Jiangmiao Pang, Jingjing Liu, Tai Wang, and Xianyuan Zhan. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations (ICLR)*, 2026.

Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.

Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 2023.

App.	Contents	Supports
A	VLA architectures, interpretability, robustness benchmarks	§2
B	Limitations and falsifiable predictions	§4.3, §4.8
C	Controls: discrete-token OpenVLA, ACT, random-direction injection	§4.3
D	SAE training, concept scoring, protocols, architectures, benchmarks, compute	§3
E	Cross-model radar and the per-architecture pathway metrics	§4.5
F	Goal-encoding dimensions, FFN neurons, layer similarity on $\pi_{0.5}$	§4.5
G	Goal steering with no instruction, on five architectures	§4.5
H	Tie-inclusive override, steering sensitivity, SAE pooling arms, DTW	§4.2–§4.6
I	GR00T phase-conditioned feature ablation	§4.6
J	Qualitative rollouts and additional figures	all
K	Per-architecture failure modes, OFT per suite, cross-model probing	§4.2–§4.4
L	Dose-response of expert injection	§4.3
M	SAE sparsity sweep and residual write-back	§4.6
N	Override under a tenfold cut in fine-tuning data	§4.3
O	UR10e setup, directed injection, displaced-object test	§4.7
P	Broader impact	

A Extended Related Work

A.1 Vision-Language-Action Models

RT-2 [Zitkovich et al., 2023] showed that web-pretrained VLMs can be fine-tuned for manipulation via tokenized actions. OpenVLA [Kim et al., 2024] extended this with an open-source 7B model, and OpenVLA-OFT [Kim et al., 2025] replaced discrete tokenization with continuous L1 regression, achieving 97.1% LIBERO success. The π_0, $\pi_{0.5}$, $\pi^*_{0.6}$, and $\pi_{0.7}$ models [Black et al., 2025, Physical Intelligence, 2025, Physical Intelligence et al., 2025, Physical Intelligence, 2026] introduced flow matching with a dedicated action expert and progressively more diverse training context. Whether $\pi_{0.7}$'s reported emergent capabilities reflect compositional generalization or further-scaled memorized motor patterns is an unexplored question that these mechanistic tools could possibly explain. Current VLAs span several scales and paradigms: SmolVLA [Shukor et al., 2025] (450M), X-VLA [Zheng et al., 2026] (cross-embodiment), GR00T N1.5 [Bjorck et al., 2025] (humanoid), CogACT [Li et al., 2024a], GR-3 [Cheang et al., 2025], VITA-VLA [Dong et al., 2025], SimpleVLA-RL [Li et al., 2026b], RIPT-VLA [Tan et al., 2025], FLOWER [Reuss et al., 2025], SpatialVLA [Qu et al., 2025], and ACT [Zhao et al., 2023] (80M, no language). Modular approaches include SayCan [Ahn et al., 2022], PaLM-E [Driess et al., 2023], and Code as Policies [Liang et al., 2023]. Concurrent representation-analysis work in robot policies uses different tools: Lei et al. [2026] apply UMAP and Wasserstein-distance analysis to characterize sim-real co-training, identifying "structured representation alignment" and "importance reweighting" effects. Our SAE-based feature-level analysis is complementary.

Action Representation. Diffusion Policy [Chi et al., 2023] treats action generation as conditional denoising, 3D Diffusion Policy [Ze et al., 2024] incorporates 3D representations, and FAST [Pertsch et al., 2025] uses DCT-based compression tokenization. Action tokenization directly determines SAE applicability (Section C.1).

Language Following. CAST [Glossop et al., 2026] augments robot datasets with counterfactual language labels. VLM4VLA [Zhang et al., 2026] found that VLM benchmark performance does not predict VLA task success, so language understanding and visuomotor competence are decoupled.

A.2 Mechanistic Interpretability

SAEs [Olshausen and Field, 1997, Bricken et al., 2023, Cunningham et al., 2024, Gao et al., 2025, Templeton et al., 2024] decompose dense activations into sparse interpretable features, with extensions to vision [Stevens et al., 2025, Joseph et al., 2025, Zaigrajew et al., 2025] and vision-language models [Pach et al., 2025]. Activation steering [Turner et al., 2023, Rimsky et al., 2024] enables behavioral control, with advances including conceptor-based steering [Postmus and Abreu, 2024], SAE-guided additions [Soo et al., 2025], and VLM safety steering [Wu et al., 2025, Sivakumar et al., 2025]. Linear probing [Alain and Bengio, 2017, Belinkov, 2022] identifies linearly decodable information, and establishing causal relevance requires intervention [Mueller, 2024]. PvP [Golovanevsky et al., 2025] shows vision and priors compete in multimodal models.

A.3 Interpretability for Robot Learning

RT-1 [Brohan et al., 2023] included attention visualizations, Diffusion Policy [Chi et al., 2023] analyzed action distributions, and ALOHA [Zhao et al., 2023, ALOHA 2 Team et al., 2024] demonstrated bimanual manipulation, all without internal representation analysis. RoboFail [Sagar et al., 2026], AHA [Duan et al., 2025], and RACER [Dai et al., 2025] characterize failures behaviorally. Häon et al. [2025] introduced VLA steering via internal representations, Molinari et al. [2025] probed for emergent world models, and Khan et al. [2025] used SAEs to isolate interpretable steering directions in Magma. Interpretability-by-design approaches include CoA-VLA [Li et al., 2025], CoT-VLA [Zhao et al., 2025], TriVLA [Liu et al., 2025], dVLA [Wen et al., 2025], and Embodied-R1 [Yuan et al., 2026].

Cross-Model and Multimodal SAE Alignment. Training SAEs on individual models, ours included, leaves each model with its own feature space. This makes it difficult to compare similar concepts across different pathways or models. There have been two directions of work that attempt to solve this. One is to scale SAEs across models. Universal Sparse Autoencoders [Thasarathan et al., 2025] and SPARC [Nasiri-Sarvi et al., 2026] train a common dictionary across models. They force concepts to be aligned by sharing an encoder or using a global TopK constraint. Group-SAE [Ghilardi et al., 2025] shares a dictionary between nearby layers based on similarity in the residual stream to reduce compute. Another direction is to align modalities. Kaushik et al. [2026] use a cross-modal random mask and a group-sparse penalty to mitigate the split dictionary effect, where features fire for only one modality. This is similar to the VLM/expert pathway split we find (§4.5), and if we trained an SAE that was aligned across modalities we could test whether the goal/motor split is orthogonal at the dictionary level or is an artifact of training each SAE separately. We leave this as future work.

Concurrent VLA Architectures. There are many VLA architectures with different methods of decoding actions that we can use our methods to study. AR-VLA [Hu et al., 2026] autoregressively generates actions with a longer memory of previous actions, allowing it to generate actions causally instead of in chunks. DFM-VLA [Chen et al., 2026] uses discrete flow matching to generate actions iteratively instead of a static set of tokens. AsyncVLA [Jiang et al., 2025] uses asynchronous flow matching and has a rater that decides when to regenerate actions. HybridVLA [Liu et al., 2026] combines both autoregressive and diffusion-based decoding in one LLM. These all change how the model will generate an action given a scene. However, our method of injecting tasks and zeroing out pathways can be applied to these architectures. We discuss our issue with where to place the hook in App. D.4. It would be interesting to see if our findings about coordinate binding hold when actions are refined iteratively, regenerated based on confidence, or generated with more context from previous actions (DFM-VLA, AsyncVLA, AR-VLA); we leave this to future work. We believe that the more an architecture re-grounds on the current observation between action chunks, with no commitment to a precomputed plan, the weaker the binding will be.

Single-Policy Diagnostics. VLA-Trace [Shi et al., 2026] traces $\pi_{0.5}$ and OpenVLA across checkpoints using kernel alignment and attention knockout. They find that the models are able to ground in the visual scene but struggle with following instructions in detail, which aligns with our findings in §4.4. LAVLA [Wulff et al., 2026] clusters the action decoder latents in GR00T N1.5 layer by layer and finds spatio-temporal and kinematic structure separating in the middle layers. Neither of these studies looks at more than 2 models, and neither intervenes on the task. Huang et al. [2026] report the same collapse of a fine-tuned $\pi_{0.5}$ on hardware when objects move. They believe this is due to drift in the visual encoder. Our probes find grounding intact (over 99% decodable), showing that the issue lies later, in the action pathway.

B Limitations

Predictions. If the expert is learning a motor program that is bound to coordinates and selected by the scene, three predictions follow. (i) Moving objects out of their training-distribution positions should cause the model to fail while keeping the instruction the same. Robustness benchmarks have shown that models struggle with position changes [Zhou et al., 2025, Fei et al., 2025]. (ii) For semantic control, one would need to change the pathway that selects the goal. Our pathway-injection results are consistent with this, since injecting into the expert causes the model to go to

the wrong location and injecting into the VLM causes it to not move. Jin et al. [2026] were unable to semantically steer the model on the feed-forward side, which we would also expect. (iii) As fine-tuning data becomes more diverse, the ability to override the model should decrease. Swann et al. [2026] show this at the feature level. But the number of demonstrations alone does not affect the ability to override (see App. N for no change in override across a $10\times$ range of demonstrations).

Benchmark Scope

All cross-architecture results come from four simulated benchmarks. The UR10e covers $\pi_{0.5}$ on one task pair (App. O) and tests injection and coordinate binding under real sensing and actuation, while visual dominance and pathway specialization remain simulation results. Fine-tuning on a new robot reshapes internal representations, so the next step is applying the same protocol to other architectures on additional embodiments. We leave this to future work, but invite the contribution to Action Atlas.

Coordinate-Frame Disambiguation

Our injection and displacement experiments (App. H.3) fit binding to absolute workspace coordinates. It could also be that the model is bound to coordinates relative to the scene that happen to line up with our source task. If we were to apply a rigid transform to the entire world in our simulation we could differentiate between these two cases, but we did not try this. Either way the model is choosing a path that is bound to coordinates and does not re-ground on the visible task. The physical displaced-object test (§4.7) separates object tracking from coordinate anchoring.

Counterfactual Prompt Coverage

The prompts that we use to test if the model is ignoring language are very basic (negation, null, swap). Stronger evidence for our language-insensitivity claims would come from more complex instructions that require the model to understand multiple objects and actions, such as "Pick up the red cup and put it behind the blue bowl", along with ambiguous multi-object scenes. There are also benchmarks such as LIBERO-PRO [Zhou et al., 2025] that test models under corrupted instructions and changes to the environment, a stricter test of language grounding.

Cross-Task Injection Confounds

When injecting into the model we are creating off-manifold activations, and if we are not careful with timing we could put the model in an invalid state. In App. C.3 we list three observations that weigh against a drift explanation. First, if we inject a random direction with the same norm as our injected activation, it is at least as far from the manifold and gives no better than chance override. Second, displacement analysis places the trajectories closer to the source task than to the destination task. Third, on hardware it is the random arm that moves incoherently (App. O).

Concept Identification

SAE features are not guaranteed to be disentangled. Ablation validation finds that some identified features encode general motor primitives and lack a clean semantic reading.

C Methodology Controls

C.1 Discrete Tokenization Prevents SAE Intervention

Before adopting OpenVLA-OFT, we conducted experiments on base OpenVLA (autoregressive discrete 256-bin tokenization). Despite successful SAE training ($R^2 = 0.87$–0.96), hooking SAEs into the forward pass produced 0% task success on all but the final layer. The discrete tokenization maps activations to bins via argmax. Even small reconstruction errors shift the selected bin, with errors compounding across the 7-token autoregressive sequence. Replacing discrete tokenization with continuous L1 regression via OpenVLA-OFT enables SAE intervention at 99.2% success. Scale is fixed at 7B, so the action representation sets SAE applicability.

C.2 ACT as Non-VLM Control

ACT is a control that does not have a language pathway to act through. There is no effect at this hook when doing cross-task injection between TransferCube and Insertion (1.0 cosine sim and all bits are the same in the action array). So ACT cannot demonstrate cross-task evidence. Grid ablations reveal spatially structured representations. We found that if we ablate the grid at index (2,2) where the majority of the task is completed we see a drop in success on both tasks (Tab. 2). Adding Gaussian noise with a std of 0.1 makes the TransferCube task fail 100% of the time and the Insertion task 75%, which shows that visual-pathway dominance holds for a vision-only policy.

C.3 Random-Direction Injection Control

An argument can be made that perturbing the residual stream enough will cause the policy to perform whatever in-distribution motor behavior it is closest to. In order to test this we used $\pi_{0.5}$, `libero_goal` and expert layers L16+L17 (the same hooks as the main cross-task condition). During each forward pass t we changed the injection $\mathbf{h}_\ell^A$ to $r_t \cdot \|\mathbf{h}_\ell^A\|_2$, a random Gaussian direction $r_t \sim \mathcal{N}(0, I)$ normalized to the unit sphere and scaled by the norm of the injection from the source task. Destination scene and source prompt remained the same. In App. L we vary how strongly we inject the directed activations.

Over the course of 50 episodes (10 pairs $\times$ 5 seeds) we were able to observe 22% source behavior, 42% destination behavior, 36% ambiguous and a success rate of 0%. Cosine similarity to source task was on average -0.017 and 0.012 to destination task. These numbers are roughly equal to 0. When we calculated our override rate (did not use a threshold, just compared cosine similarity to source vs dest) we achieved 50% [36.6–63.4], which is comparable to chance.

When performing our cross-task injection we achieved a 99.6% source behavior rate using the same test suite and layers (1,614 episodes), a difference of 78pp with no overlap in confidence intervals. This shows that the injected direction is what allows for overriding. Injecting a random direction with the same norm only throws the policy off.

Replication at n=200. With the destination prompt held on its own task text (10 pairs $\times$ 10 episodes per suite, with seeds starting at 42), we see a 48.0% override rate on `libero_goal` and 43.0% on `libero_object`. Pooled together this is 45.5% (200 episodes), consistent with the 50% chance level. Mean cosine similarity to the source trajectory was 0.01 and to the destination trajectory was 0.02 on both suites. Success rate on the destination task was 30% on goal and 0% on object, compared to 95% and 92% respectively with no injection. So we can throw off the policy with a random direction but not cause it to perform the source task.

Off-manifold distance does not predict override. Injected source activations are off-distribution by construction. We take a Gaussian fit of the expert layers on 6 `libero_goal` tasks (8k samples per layer) and score our held-out natural samples against our injected samples from the other 4 tasks. We get a Mahalanobis distance of 46.9 on injected samples vs 18.0 on natural samples at L16 (median distances of 40.7 and 17.5). Only 0.4% of injected samples fall within the 95th percentile of natural samples. At L17 we get a distance of 34.4 vs 16.7 with 6.5% of injected samples falling within the 95th percentile. If we include the source tasks in our distribution we see the gap decrease (L16 32.9 vs 20.0 with 9.1% within the 95th percentile; L17 24.8 vs 17.9 with 38.9% within the 95th percentile). Our random direction is at least as far away from the natural distribution and still gives chance-level override. So being far away from the natural distribution is not what is causing our override. If the policy were just drifting off manifold we would expect to see incoherent movement. When we inject the source direction we see the policy produce a complete source-task trajectory and move towards the coordinates of the source task. The random direction on the UR10e is what causes the policy to move incoherently (App. O).

D Extended Methodology Details

D.1 SAE Architecture and Training

The 520+ SAEs of Tab. 7 are $\pi_{0.5}$ 72 (18 layers $\times$ 2 pathways $\times$ 2 pooling), OFT 96 (32 layers $\times$ 3 arms), X-VLA 120 (24 layers $\times$ 2 pooling $\times$ 2 environments, plus temporal), SmolVLA 146 (32

layers × 2 pathways × 2 pooling, plus 18 temporal), and GR00T 87 (32 layers × 2 pooling, plus 23 temporal). Concepts total 79 unique names, or 82 if the three names shared by the spatial and object categories count once per category.

Architecture. Following the TopK design of Gao et al. [2025], our sparse autoencoders consist of an encoder-decoder pair with tied weights. For an input activation $\mathbf{h} \in \mathbb{R}^d$ (per token; $d \in \{480, 960, 1024, 1536, 2048, 4096\}$ across the five VLAs),

$$\mathbf{z} = \mathrm{TopK}_k(\mathbf{W}_e(\mathbf{h} - \mathbf{b}_d)), \tag{3}$$

$$\hat{\mathbf{h}} = \mathbf{W}_e^\top \mathbf{z} + \mathbf{b}_d, \tag{4}$$

where $\mathbf{W}_e \in \mathbb{R}^{m \times d}$ with expansion factor $m \in \{4d, 8d\}$, and $\mathrm{TopK}_k(\mathbf{u})$ retains the $k = 64$ largest pre-activations and zeros the remainder. Tied decoder weights ($\mathbf{W}_d = \mathbf{W}_e^\top$) and the pre-encoder bias subtraction follow Bricken et al. [2023], Gao et al. [2025].

Training objective. TopK allows us to enforce sparsity without needing to add an L1 penalty [Cunningham et al., 2024]. We optimize reconstruction MSE plus an AuxK term [Gao et al., 2025]. AuxK recycles dead features by reconstructing the residual $\mathbf{r} = \mathbf{h} - \hat{\mathbf{h}}$ (input − reconstruction) using the top k_{aux}=512 inactive features:

$$\mathcal{L}(\mathbf{h}) \;=\; \underbrace{\|\mathbf{h} - \hat{\mathbf{h}}\|_2^2}_{\text{reconstruction}} \;+\; \alpha_{\text{aux}}\; \underbrace{\|\mathbf{r} - \mathbf{W}_e^\top \mathrm{TopK}_{k_{\text{aux}}}(\mathbf{z}_{\text{dead}})\|_2^2}_{\text{AuxK (dead-feature recycling)}}, \tag{5}$$

with $\alpha_{\text{aux}} = 1/32$. Decoder columns are unit-normalized after each step.

Training Hyperparameters. Each SAE was trained on 500k activations (∼10k forward passes × 50 action tokens) with a batch size of 4096 for 100 epochs. The learning rate was 3×10^{-4} with cosine decay. We used TopK to select k=64 active features per token. Decoder weights were tied to the encoder transpose ($\mathbf{W}_d = \mathbf{W}_e^\top$).

D.2 Concept-Based Feature Identification

Features associated with concepts were selected using the frequency-weighted contrastive score described in §3.4 ($\text{score}_f = d_f \times \text{freq}_f$). The active set is the top 64.

With TopK sparsity, features with high mean activation across samples do not necessarily appear in the active top-64 for individual samples, reducing their causal relevance.

Sample population for Cohen's d. Activations were max-pooled over the 50 action tokens in each episode, giving us an activation vector for each (episode, layer, feature). Cohen's d_f was calculated over episodes in each concept set (n=100–200 episodes per concept). We take the difference in mean between episodes that have the concept and those that do not and divide by the std. We use unbiased Welch variance for our population stats.

Concept specificity scores in Tab. 14(a). The dimensionless score $\text{score}_f = d_f \times \text{freq}_f$ is bounded above by $|d_f|$ and is at most a few units of effect size in our data. The values reported in parentheses in Tab. 14(a) (e.g., 133k, 412k) are raw cumulative activation magnitudes per concept, computed as $\sum_{(\text{ep},t)\in\mathcal{C}} \mathrm{ReLU}(z_{f,t,\text{ep}})$ over the concept-positive episode set $\mathcal{C}$ and the 50 action-token positions, summed across layers. We retain the raw-magnitude column for readability of layer rankings within a concept. The per-feature dimensionless score_f used to identify concept-associated features is reported in the supplementary feature-exploration platform.

D.3 Ablation Protocol

Feature ablation is performed by zeroing selected features in the SAE latent space:

$$\mathbf{z}_{\text{ablated}} = \mathbf{z} \odot \mathbf{b} \tag{6}$$

$$\mathbf{h}_{\text{modified}} = \mathbf{h} + \mathbf{W}_e^\top(\mathbf{z}_{\text{ablated}} - \mathbf{z}) \tag{7}$$

Table 6: Architectures under study. PG = PaliGemma, Exp. = Expert, Flor. = Florence, D+E+V = DiT + Eagle + VL-SA, LIB. = LIBERO, MW = MetaWorld, SimpE. = SimplerEnv. Details in App. D.5.

Model	Params	Layers	Dim	Action Gen.	Pathway	Bench.
$\pi_{0.5}$ [Physical Intelligence, 2025]	3B	18	1024	Flow (10 steps)	Dual (PG + Exp.)	LIBERO, UR10e
OFT [Kim et al., 2025]	7B	32	4096	Cont. L1 regr.	Single (Llama-2)	LIBERO
X-VLA [Zheng et al., 2026]	∼1B	24	1024	Flow matching	Single (Flor.-2)	LIB., SimpE.
SmolVLA [Shukor et al., 2025]	450M	32	960/480	Flow matching	Dual (VLM + Exp.)	LIB., MW
GR00T [Bjorck et al., 2025]	3B	32	varies	Diff./flow	Triple (D+E+V)	LIBERO
ACT [Zhao et al., 2023]	80M	–	–	CVAE	Enc.-Dec.	ALOHA

where $\mathbf{b} \in \{0,1\}^m$ is a binary mask (with m the SAE expansion dimension) carrying zeros at ablated feature indices. Per-token ablation applies this independently to each of the 50 action tokens. Writing the modification as a residual ($\mathbf{W}_e^\top(\mathbf{z}_{\text{ablated}} - \mathbf{z})$) preserves the unmodeled component of the activation and intervenes only on what the SAE explains.

Steering operator. The ablation operator generalizes to a continuous steering operator that modulates a target set of features $\mathcal{F}$ by a scalar coefficient α:

$$\mathbf{z}_{\text{steered},f} \;=\; \begin{cases} \alpha\,\mathbf{z}_f & f \in \mathcal{F}, \\ \mathbf{z}_f & \text{otherwise}, \end{cases} \qquad \mathbf{h}_{\text{steered}} \;=\; \mathbf{h} + \mathbf{W}_e^\top(\mathbf{z}_{\text{steered}} - \mathbf{z}). \tag{8}$$

$\alpha = 0$ recovers ablation, $\alpha = 1$ is identity, $\alpha \in (0,1)$ dampens, $\alpha < 0$ negates, and $\alpha > 1$ amplifies. Because TopK enforces hard sparsity, $\alpha\,\mathbf{z}_f = 0$ whenever feature f is inactive in the top-k set. This multiplicative operator therefore modulates only currently-active concepts and cannot inject features absent from the active set. Additive steering ($\mathbf{z}_f \leftarrow \mathbf{z}_f + \alpha\,\mathbf{v}_f$ for a steering vector $\mathbf{v}_f$) injects concepts outside the active set, and App. G applies it in the VLM pathway. The cross-architecture sensitivity Fig. 7 reports ablation ($\alpha = 0$).

D.4 Intervention-Hook Protocol

Activation interventions hook one of two locations: the full *layer output* (residual stream after attention + MLP + residual addition) or, where the layer wrapper is bypassed, the *MLP output* that writes into the residual stream. Some VLAs bypass the standard DecoderLayer in their inference path, so hooking the layer wrapper does not modify what the policy consumes.

Per-architecture hook placement. All interventions hook the layer output (i.e., full residual stream): cross-task injection (Eq. (1)), layer zeroing, SAE training, SAE reconstruction and ablating/steering SAE features (Eqs. (7) and (8)), meaning each SAE will act on the activations it was trained on. SmolVLA is an exception because its interleaved VLM-expert loop invokes the submodules of each layer directly and does not call the wrapper `DecoderLayer.forward`. So we hook `layer.mlp.forward`, which is the MLP writing to the residual stream, for all these interventions. Goal-steering interventions from App. G directly affect the residual stream of SmolVLA by reading out the post-attention residual using a pre-hook on the layer's post-attention norm, adding the MLP output and writing out using the MLP output. We confirmed hook visibility using cosine identity ($\cos = 1.0$ between hooked and unhooked baselines when doing no-op interventions).

Reading the full-layer interventions. The OFT 14–15% recovery under null prompt plus baseline injection in Tab. 2, and the X-VLA and GR00T 0% under single-layer zeroing, replace the full layer output, so they are upper bounds on layer importance under residual-stream patching. On SmolVLA the same interventions change only the MLP write and leave the residual passthrough intact.

D.5 Model Architecture

Pi0.5 Architecture

$\pi_{0.5}$ uses PaliGemma (3B parameters) as its vision-language backbone and an 18-layer Gemma transformer (1024 hidden dimension) as the action expert. The action space is 7-dimensional (dx, dy, dz, dax, day, daz, gripper). The model generates 50 actions per forward pass through flow matching with iterative denoising.

Table 7: We performed experiments at scale on 6 models. Number of episodes is the number of rollouts summed over every experiment type the model participated in. Number of SAEs includes all per-token, mean-pool and temporal arms (see App. D.1). There is some overlap between models when counting concepts, and the total is the union of all concept names. ‡ We only kept aggregate statistics for large ablation sweeps. Our released dataset contains per-episode data for 394,000+ episodes.

Model	Episodes	SAEs	Concepts
$\pi_{0.5}$	71,500+	72	46
OpenVLA-OFT	70,700+‡	96	45
X-VLA	74,500+	120	55
SmolVLA	58,000+	146	60
GR00T N1.5	164,700+	87	39
ACT	1,870	–	–
Total	**>420,000**	**520+**	**79**

OpenVLA-OFT Architecture (Continuous)

OpenVLA-OFT is similar to OpenVLA in that it uses the same Llama-2 7B backbone and Prismatic vision encoder (DINOv2 and SigLIP). It differs by using continuous action regression instead of discrete tokens. It has an MLPResNet action head, trained with L1 loss, that predicts all 7 dimensions of the action, and it outputs 8-step action chunks (56 action tokens) in one forward pass. Optimized Fine-Tuning (OFT) uses LoRA to fine-tune the model while preserving the pretrained representation geometry.

X-VLA Architecture

X-VLA (0.9B parameters) encodes the main camera view and the instruction with Florence-Large and generates actions with a 24-layer transformer (1024 hidden dimension) trained by flow matching. Learnable soft prompts, one set per data source, condition the shared transformer on the embodiment, and domain-specific output projections map its tokens to action chunks.

SmolVLA Architecture

SmolVLA (450M parameters) pairs a SmolVLM2 vision-language model (960 hidden dimension) with a flow-matching action expert at half that width (480). The LIBERO checkpoint we use keeps all 32 VLM layers and a 32-layer expert. The two run layer by layer: expert layers alternate cross-attention to the VLM's keys and values with causal self-attention over the action tokens. The expert generates 50-action chunks with 10 denoising steps.

GR00T N1.5 Architecture

GR00T N1.5 (3B parameters) has three pathways. A frozen Eagle vision-language model supplies the output of its 12th LLM layer, 4 vision-language self-attention (VL-SA) layers process those tokens, and a 16-layer diffusion transformer (DiT) generates actions. DiT blocks alternate cross-attention to the vision-language tokens with self-attention over state and action tokens, and produce 16-step action chunks with 4 flow-matching steps.

ACT Architecture

ACT uses a ResNet-18 vision encoder (pretrained on ImageNet) feeding into a Transformer Encoder-Decoder with a CVAE latent space ($\beta = 10$, latent dimension 32). The encoder processes multi-view camera observations, and the decoder generates action chunks of 100 timesteps in 14-DOF joint space (7 per arm for bimanual manipulation).

D.6 Benchmarks

LIBERO [Liu et al., 2023] comprises four suites of 10 tasks each: **Goal** (different goals in a shared scene), **Object** (pick-and-place with varied objects), **Spatial** (spatial reasoning and relational

placement), and **LIBERO-10** (diverse tasks spanning all three categories). MetaWorld [Yu et al., 2020] provides 50 tabletop manipulation tasks grouped by difficulty (easy, medium, hard, very hard). SimplerEnv [Li et al., 2024b] evaluates sim-to-real transfer on Google Robot and WidowX embodiments. ALOHA [Zhao et al., 2023] tests bimanual manipulation with the ACT policy.

D.7 Compute Requirements

Experiments were conducted on an 8×A100-SXM4-80GB cluster for $\pi_{0.5}$, OpenVLA-OFT, large-scale SAE training, concept ablation, and cross-model analysis, an NVIDIA RTX 5090 (32GB) for GR00T N1.5 experiments, and two NVIDIA RTX 4090s (24GB) for SmolVLA and X-VLA experiments. Total experiment data exceeds 7.1 TB, including 4.3 TB of activation recordings, 152 GB of SAE checkpoints, and 420,000+ rollout episodes. The full study used approximately 10,000 A100 GPU-hours.

D.8 Evaluation Protocol

Episode counts per condition are stated with each experiment (20 to 50 per task for the main results, 5 per task for the earliest sweeps, and 3 per task for the OFT goal-steering screen). Episodes run for a maximum of 300 steps.

E Cross-Architecture Synthesis

Figure 4 condenses the cross-architecture results of §4.2–§4.6 into one view across six axes:

- **Baseline success:** the suite mean of the baseline rows in Tab. 2.
- **Visual override:** destination success lost under cross-task injection, 100 − Dst. SR in Tab. 3.
- **Language tolerance:** null-prompt over correct-prompt success, averaged over the LIBERO rows of Tab. 4 (100 means policy ignores prompt).
- **SAE fidelity:** best-arm rollout success as a fraction of baseline (Tab. 12).
- **Cross-task transfer:** the tie-inclusive override rate (Tab. 10). SmolVLA's value comes from Tab. 3, since its tie fraction is 0%.
- **Ablation tolerance:** the zero-effect rate of single-feature ablation: $\pi_{0.5}$ 54%, OFT 92%, X-VLA 82%, SmolVLA 28%, GR00T 59% (Fig. 7).

Per-architecture detail is in App. K, and the SAE pooling results are in App. H.

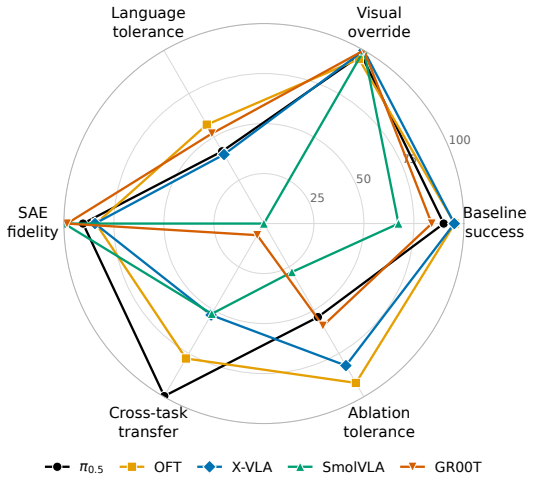


Figure 4: **Cross-model summary.** Capability radar across the five language-capable VLAs. Each axis maps a table value linearly onto 0–100, with axes defined in App. E.

The pathway-specialization claim of §4.5 rests on three architecture-specific metrics, since no single metric applies to all three multi-pathway models.

F Component-Level Analysis

This section traces information flow below the layer level: individual activation dimensions, FFN neurons, and layer-to-layer representational similarity. We analyze $\pi_{0.5}$ in depth (FFN weight projection and LDA on the 1024-dim expert space), then validate across OFT (32-layer neuron-level concept mapping, 80 concepts $\times$ 11,008 neurons per layer), SmolVLA (FFN neuron contrastive identification on both expert and VLM pathways), and GR00T (per-layer-type SAE feature utilization across DiT, Eagle, and VL-SA).

F.1 Goal Encoding Dimensions

Linear discriminant analysis (LDA) on $\pi_{0.5}$ expert activations at layer 17 identifies three components that separate four goal-suite tasks with 71.9% cross-validated accuracy (n=114, 5-fold). The first component captures 45.8% of discriminant variance, the second 35.2%, and the third 19.0%. The top 20 discriminant dimensions (417, 909, 934, 649, 147, 219, 708, 297, 155, 545, ...) define a low-dimensional subspace sufficient for goal identification within a 1024-dimensional activation space.

Goal information emerges at different rates in different pathways. For PaliGemma we see the ability to classify goals increase from 56.4% at layer 0 to 76.4% at layer 13 and then drop off to 68.9% at layer 17. The expert shows a flatter trend, at 64.0% for layer 0, with a peak of 66.4% at layer 9 and ending at 62.6% for layer 17. PaliGemma's steeper rise is consistent with its role as the goal encoder. The expert seems to have some knowledge about the goal throughout but never refines it.

Individual activation dimensions participate in both goal encoding and action generation. Dimension 62 most strongly modulates the x-component of action output (score 0.074), dimension 618 modulates y (0.071), and dimension 14 modulates z (0.066). The overlap between goal-discriminant dimensions (417, 909, 934) and action-modulating dimensions (62, 618, 14) is minimal, consistent with separable subspaces for goal identity and motor execution (tested below).

Subspace separability via subspace injection. We test separability by injecting only the goal-discriminant subspace (20 of 1024 dimensions from LDA) from task A into task B's forward pass at layer 17. Across five task pairs on `libero_goal`, full injection (all 1024 dimensions) causes complete task failure (0% success on 4/5 pairs, down from 100% baseline). Goal-subspace injection (20 dimensions, 2% of the activation space) preserves task success: 100% on three pairs, 67% on a fourth where the source and destination share the OPEN motor primitive. Action-subspace injection (15 dimensions encoding x/y/z action components) shows the same pattern. Replacing 2% of the activation space from a different task does not disrupt the destination task's motor execution, evidence for separability of goal identity from motor execution within the same layer, and stops short of direct causal control over goal selection. Additive steering along goal directions in the VLM pathway is tested in App. G.

F.2 FFN Motor Primitive Neurons

Projecting PaliGemma FFN weight vectors onto a token-association vocabulary identifies neurons that encode specific motor primitives. At layer 17, 6,606 of 16,384 neurons (40.3%) associate with motion verbs (pick, place, move, push, pull), 2,907 (17.7%) with object tokens, 2,989 (18.2%) with gripper state, 2,223 (13.6%) with direction, and 1,151 (7.0%) with spatial relations.

As we move through the layers we see the share of these neurons change. Motion verb neurons go from 4,231 at layer 0 (25.8%) to 6,606 at layer 17 (40.3%), a 56% increase. Gripper neurons triple from 998 (6.1%) to 2,989 (18.2%). Object neurons decrease from 3,660 (22.3%) to 2,907 (17.7%), and spatial neurons decrease from 1,655 (10.1%) to 1,151 (7.0%). Going from objects/spatial to motion/gripper as we move through the layers shows how the model goes from what it sees to what it should do within one pathway.

OFT's 32-layer Llama-2 shows different characteristics. Concept mapping at finer granularity (80 concepts per layer across 11,008 neurons) shows spatial relation neurons dominating at every depth (`spatial_in`: 5,483 $\rightarrow$ 7,245; `spatial_on`: 4,724 $\rightarrow$ 6,727). These grow monotonically from layer 0 to layer 31, while neurons representing objects stay sparse (e.g., `obj_can`: 814 $\rightarrow$ 1,175). Consistent with OFT's resilience to concept ablation (92% of ablations had no effect), spatial

information is spread across the layers of its 4096-dimensional representation. GR00T's SAE features show a gradient in feature use by layer type, with later DiT layers using more features than earlier ones. At DiT layer 0 only 3,654 out of 12,288 features are alive (70% dead). By DiT layer 15 this number increases to 9,317 alive features. Eagle layer 0 has 6,730 alive features out of 16,384 and VL-SA layer 0 has 11,585 alive features.

F.3 Layer Similarity via CKA

Linear Centered Kernel Alignment (CKA) between $\pi_{0.5}$ expert layers 5 and 11 on LIBERO-Goal, computed on paired samples (the same rollout steps and token positions in both layers), is 0.79. A shuffled-row control gives 0.02. The expert therefore carries a shared residual representation that later layers refine, consistent with injected activations persisting through the remaining layers.

When looking at the probes on the 32 layers of the Llama-2 backbone used in OFT we see similar behavior. R^2 for predicting episode length increases from 0.845 at layer 0 to 0.941 at layer 24 then decreases to 0.915 at layer 31. Probes for classifying the task reach 97.7–100% by layer 8. This, along with the previous results on the FFN layers, indicates that both architectures have layers that refine a shared representation.

F.4 Where Spatial Binding Forms During Fine-Tuning

Comparing backbone activations between base OpenVLA (fine-tuned with discrete action tokens) and OFT (fine-tuned with continuous L1 regression) on the same LIBERO tasks localizes the representational changes induced by fine-tuning. We feed identical images through both models and measure per-layer cosine similarity of mean activations (n=50 images per suite, 4 suites).

We find that as we go deeper into the network the divergence increases, across all four suites. L1–L15 have cosine similarities of >0.999. L16–L28 range from 0.997 to 0.980 and the last layer L31 is 0.917–0.938. Layer 0 is slightly lower at 0.970–0.989 due to the difference in input embeddings.

The long-horizon `libero_10` suite diverges most at L31 (0.917), and the three single-concept suites least (0.935–0.938).

Fine-tuning therefore reshapes the final third of the backbone and leaves early and middle layers intact. The coordinate-bound programs of §4.3 sit in the layers that change most.

Four OFT models fine-tuned on different suites (6 pairs $\times$ 32 layers, n=30 images each) differ much less at L31 (0.973–0.992) and not at all in the middle (L16: 0.998–0.999). The action head (discrete against continuous) reshapes late layers more than the choice of fine-tuning suite.

G Goal Steering in the VLM Pathway

§4.5 places goal selection in the VLM pathway. Here we test that claim causally by asking whether an edit there, with no instruction present, makes the policy complete a chosen task in a scene that holds several possible goals.

Protocol. We replace the instruction with a filler string of function words that have the same number of tokens, so all other tokens in the prompt keep their positions. We take the residual stream from the VLM pathway when given the true instruction and the filler instruction, with 3 rollouts for each. Only the instruction tokens are edited (and all tokens after the instruction for causal backbones). We try 3 types of edits. (i) A *patch* replaces the edited tokens with their mean under the true instruction. (ii) A *per-position gap* adds the per-position mean difference between the true instruction and filler. (iii) A *single direction* adds a single unit vector, the per-position difference averaged over positions, scaled to a fraction ρ of each token's norm. We use a random direction with the same norm and a gap from another task as controls. Filler and true instruction with no edit act as our floor and ceiling.

Result. By editing one layer in the VLM pathway we are able to restore goal selection with no instruction on $\pi_{0.5}$, SmolVLA, X-VLA and GR00T. When we edit with a random direction or a direction from another task of the same size we do not see this effect (Tab. 8). OpenVLA-OFT does not show this behavior. On $\pi_{0.5}$ we can complete 113/150 episodes with one direction at layer 2 of PaliGemma, compared to 6/150 with a random direction, and 9 out of 10 tasks show this behavior.

We found layers that work (2 and 5) after failing to find any at layers 9 and 13 in our initial pilot, so the instruction is read early in the VLM stack.

The form of the signal depends on the architecture. $\pi_{0.5}$ carries the goal as one direction shared across instruction positions. SmolVLA carries it per position, so one averaged direction fails there at every magnitude. X-VLA's bidirectional Florence encoder spreads the instruction into the image tokens, so the edit has to cover the whole Florence block. GR00T responds to the per-position gap on its Eagle backbone at its ceiling. OpenVLA-OFT runs image and instruction through one Llama stack with bidirectional attention, and the per-position gap over the whole sequence at layer 4 or 12 completes 2/24 episodes on the eight tasks without a filler floor, against 24/24 for the true instruction.

Table 8: Goal steering with the instruction replaced by a length-matched filler, as successes over episodes for the chosen task. $\pi_{0.5}$, GR00T (Eagle L6) and OFT (Llama L12, public checkpoint, official harness) run 10 `libero_goal` tasks. X-VLA runs 9, excluding one task that succeeds under the filler. SmolVLA runs 5 `libero_object` tasks.

	$\pi_{0.5}$	SmolVLA	X-VLA	GR00T	OFT
Filler, no edit	12/150	0/50	2/90	3/100	6/30
True instruction	139/150	46/50	86/90	91/100	30/30
Patch (instruction states)	137/150	44/50	–	–	–
Per-position gap	–	26/50	50/90	93/100	8/30
Single direction	113/150	0/150	–	–	–
Random, matched norm	6/150	0/150	3/90	10/100	6/30
Other task's edit	2/150	1/150	0/90	0/100	0/30

Pooling and SAE features. We use the same setup to see if SAEs preserve the signal we can steer with (Tab. 9). For X-VLA and GR00T we have per-token and mean-pooled SAEs on the layers we edit, and we add the per-position gap after applying each SAE. The per-token SAE reconstructs each position and the mean-pooled SAE reconstructs the mean of the positions, since that is what it was trained to do. On X-VLA we find that the per-token SAE can steer just as well as the gap itself (54/90 vs 50/90 at layer 0, Fisher p=0.65). The mean-pooled SAE falls to the random floor (2/90, p=7.5×10^{-19} compared to per-token). If we mean pool the raw gap without any SAE we already lose most of the ability to steer. We see similar results on GR00T. §4.6 shows that X-VLA and the action head on GR00T can be mean pooled when reconstructing rollouts, but the instruction signal in their language pathway requires the per-token SAE. So pooling preference also depends on which signal the SAE needs to keep. On $\pi_{0.5}$ we can create a direction using the top 32 features from the per-token PaliGemma SAE and get 64/150 at layer 5 compared to 74/150 with the dense direction. SmolVLA's SAEs are trained on the output of MLPs, which do not contain the instruction in earlier layers (patching these alone gets us 0/50), so we cannot perform this test.

Table 9: Steering through SAE reconstructions. The per-position instruction gap is added after reconstruction by each SAE, with n=90 (X-VLA) and n=100 (GR00T) per cell.

	X-VLA L0	X-VLA L2	GR00T L2	GR00T L6
Raw gap	50	48	94	93
Per-token SAE	54	46	66	88
Raw gap, pooled over positions	14	10	9	24
Mean-pool SAE, per position	13	6	8	17
Mean-pool SAE on the pooled gap	2	3	4	10
Random, matched norm	3	0	5	10

Scope. The edit is computed from rollouts under the true instruction in the same scene, so it restores a goal the policy already executes. It supplies a goal where no instruction is present and does not test overriding a competing instruction. Success rates are in simulation, with 3 (OFT) to 15 ($\pi_{0.5}$) episodes per task for each edit and magnitude. Rows that pool magnitudes (SmolVLA single direction and controls) sum over them.

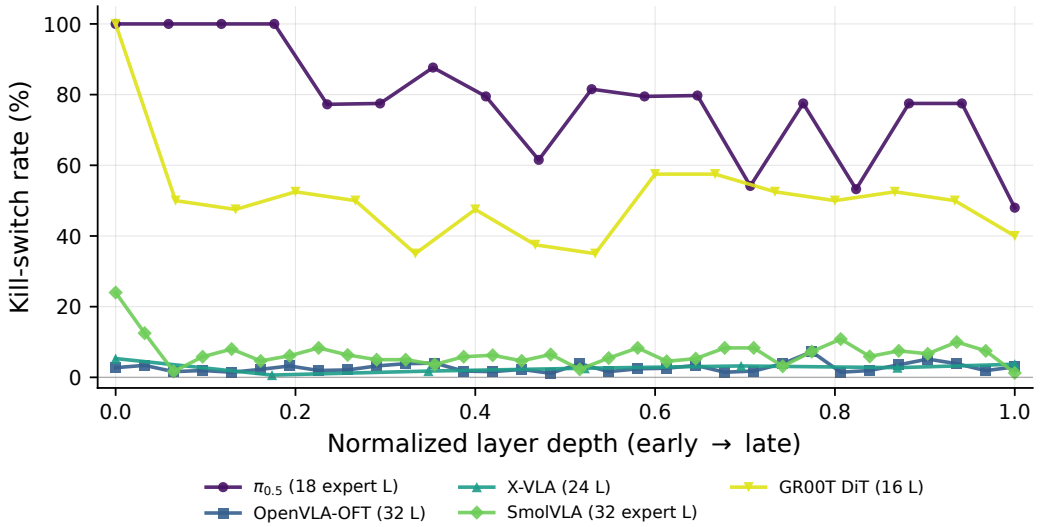


Figure 5: Per-layer kill-switch profile: the fraction of (concept, task) ablation pairs with $\Delta\text{SR} \geq 0.5$, against normalized layer depth. Pair counts are in Fig. 7.

H Additional Experimental Results

Tie-inclusive override rates. Table 10 reports override without the tie-band exclusion of Eq. 2, i.e., $\Pr[\Delta_t > 0]$ over all injection steps, for every model with saved trajectories. The $\pi_{0.5}$ and OFT rows use the action-vector-cosine metric of the main text, and the X-VLA and GR00T rows use the end-effector-velocity classifier. X-VLA under action-vector cosine including ties is 39.5%. Cross-architecture comparisons should be read within one metric.

Table 10: Tie-inclusive override ($\Pr[\Delta_t{>}0]$). n counts episodes with saved trajectories, so it can be smaller than in Tab. 3. *SmolVLA's per-episode classifier has no tie band, so its tie-inclusive and conditional rates coincide.

Model	n	Tie-incl. override [95% CIs]	Tie fraction	Metric
$\pi_{0.5}$	1,968	99.6% [99.2, 99.8]	0.3%	action cosine
OFT	1,079	77.9% [75.4, 80.3]	20.9%	action cosine
X-VLA	2,907	52.8% [51.0, 54.6]	32.7%	EEF-velocity cosine
GR00T	180	6.7% [3.9, 11.3]	5.6%	EEF-velocity cosine
SmolVLA	720	52.1% [48.4, 55.7]	0.0%*	per-episode override

$\pi_{0.5}$ null-prompt recovery and batch agreement. The $\pi_{0.5}$ null and injection rows of Tab. 2 come from one matched batch (10 tasks $\times$ 5 seeds, n=50/suite, standard chunked action execution). Against that batch's own baselines (90.0%, 92.0% and 88.0% on goal, object and spatial), injection recovers 88%, 69% and 62% of the baseline-to-null gap (Fisher $p{=}1.1{\times}10^{-14}$, $2.6{\times}10^{-6}$ and 0.13, injection against null). The n=500 sweep in Tab. 4 (31.6% object, 5.8% goal, 55.0% spatial) agrees with it on all three suites (object and goal inside the n=50 Wilson intervals; spatial 31/50 vs. 275/500, Fisher p=0.37).

Steering sensitivity. Models vary in their sensitivity to feature steering. $\pi_{0.5}$ expert pathways fail when suppressing features ($\Delta\text{SR} = -84$pp at $-3\times$) and GR00T DiT fails when amplifying features (-68pp at $9\times$), while OFT (-6pp at $-3\times$) and SmolVLA (-3pp at $5\times$) can handle these amounts of steering. We believe this is due to the sub-millimeter precision needed in manipulation tasks [Chi et al., 2023], errors compounding over the 50 tokens of an action chunk [Bengio et al., 2015], and no closed-loop correction in behavior-cloned policies [Ross et al., 2011]. Steering during different task phases on $\pi_{0.5}$ shows that the model's sensitivity is localized to the transport phase ($p = 0.013$, two-proportion z-test).

H.1 Per-Token vs Mean-Pooled SAE Reconstruction

Fig. 6 reports the 18-layer × 2-pathway × 2-pooling-mode rollout validation on $\pi_{0.5}$/`libero_object` (n=20 episodes per (layer, pooling) cell; 1,600 episodes in total). The two SAE families are trained on success-only baseline-rollout activations under identical hyperparameters (k=64, 8× expansion, 30 epochs), differing only in whether the latent code is computed per token or after mean-pooling and broadcast back to every position.

Fig. 12 shows cross-model SAE reconstruction. Mean-pool destroys the $\pi_{0.5}$ expert at all layers, dropping from a baseline of 93.8% to 8.9% when averaged over all 18 layers. Per-token maintains success at all expert layers (range 55–100%, with an average of 83.6% over 18 layers). For the PaliGemma pathway we see the opposite happen at the end of the stack. Mean-pooled SAEs are able to maintain 60–85% success at layers 14–17 while failing at earlier layers (0% success at layers 0–12 and 5% at layer 13). This is consistent with PaliGemma's role as a goal/scene encoder whose late-layer activations do not vary by position and so tolerate pooling. We also compare each task in LIBERO-10 for the $\pi_{0.5}$ expert (n=250 for each task) and find similar but more pronounced results: the per-token SAE achieves 1,748/2,500 successes (69.9%) and mean-pool 9/2,500 (0.4%).

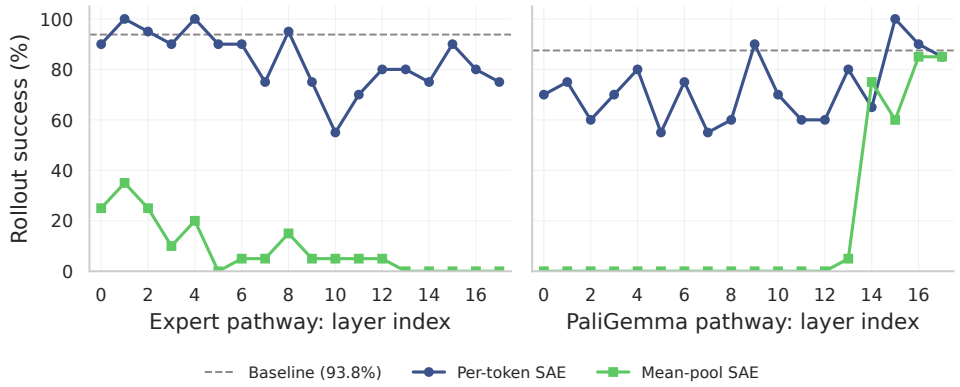


Figure 6: Per-layer rollout success through per-token and mean-pool SAEs on $\pi_{0.5}$ `libero_object` (n=20/cell). Dashed lines: per-pathway baselines (expert 93.8%, PaliGemma 87.5%).

H.2 Temporal-Contrastive SAE Pooling (T-SAE Ablation)

Does adding a temporal-aware SAE that operates per token but pushes neighboring codes to be closer together close the per-token vs. mean-pool gap? We take the Temporal SAE (T-SAE) recipe from Bhalla et al. [2026] and apply it to VLA action tokens. Using this as a third pooling arm, we compare to the per-token and mean-pool results from Fig. 6 with the same activations. TopK ($k = 64$), expansion ($8\times$), optimizer and number of epochs are all the same. See Training details below for the loss and other training settings.

Pooling preference is architecture-tied. T-SAE checkpoints are trained on five architectures ($\pi_{0.5}$ expert 18 layers, X-VLA 24, SmolVLA expert 32, GR00T-DiT 16, OFT 32) and validated by rollout on all five (Tab. 12). Two regimes appear. On $\pi_{0.5}$ (n=1,440 per arm, Tab. 11) and OFT (Tab. 17), mean-pooling collapses rollout success, so both models carry position-distinct phase information in the action-token residual stream. X-VLA (n=4,800 per arm), SmolVLA, and GR00T-DiT carry close to position-invariant action-token activations and tolerate mean-pooling, with SmolVLA indifferent within 0.8 pp. OFT's similarity to $\pi_{0.5}$, despite a different VLM and action head, places the divider in the residual-stream encoding. The temporal-contrastive arm degrades both regimes (mechanism below).

Mechanism. We run a diagnostic on layer 8 (1.5M pairs of adjacent timesteps with 5K held out) to see how close the codes $z = \text{encode}(\mathbf{h})$ of adjacent timesteps are to one another. Per-token and mean-pool have $\cos(z_t, z_{t+1})$ of 0.857 and 0.852 respectively, meaning that pooling does not affect

Table 11: $\pi_{0.5}$ expert 3-arm SAE pooling (n=1,440/arm, 18 layers $\times$ 4 suites $\times$ 20 ep). Deltas vs. Tab. 2 baseline.

Suite	Baseline	Per-token	Temporal	Mean-pool
libero_goal	93.2%	83.1% (–10.1)	76.1% (–17.1)	8.1% (–85.1)
libero_spatial	92.8%	**90.0%** (–2.8)	**88.3%** (–4.5)	1.9% (–90.9)
libero_object	87.6%	81.9% (–5.7)	74.7% (–12.9)	2.2% (–85.4)
libero_10	86.0%	70.3% (–15.7)	46.7% (–39.3)	1.1% (–84.9)
Mean (4 suites)	**89.90%**	**81.32%**	**71.46%**	**3.33%**

this property and its damage comes at inference. The temporal-contrastive variant is able to increase this to 0.997, which is the optimal value for its InfoNCE loss. Because adjacent timesteps already share most of the same active features (TopK Jaccard of 0.61–0.71 compared to 0.12–0.18 for random pairs), the contrastive loss only needs to make the coefficients similar. This causes the adjacent timestep codes to become nearly identical and lose phase information that the policy uses later.

Cross-architecture confirmation. The contrastive arm converges to $\cos_{\text{adj}} \approx 0.997$ on $\pi_{0.5}$, X-VLA, SmolVLA, and GR00T-DiT (SmolVLA 0.940, slightly under-converged) while keeping per-token-grade R^2 (≥ 0.97). On OFT, training is data-limited ($\sim$26K pairs/layer vs. $\geq$ 100K elsewhere) and $\cos_{\text{adj}}$ rises only to 0.717. Per-architecture per-token reconstruction means and InfoNCE values are folded into Tab. 12.

Table 12: Rollout fidelity of the three SAE arms per architecture, with T-SAE training metrics under matched hyperparameters (α=1.0, k=64, 8$\times$). **Bold**: best arm per row.

Model (pathway)	L	Baseline	Per-token	Temporal	Mean-pool	Recon (PT)	Recon (T)	InfoNCE
$\pi_{0.5}$ (expert)	18	89.9%	**81.3%**	71.5%	3.3%	0.018	0.026	5.26
X-VLA (transformer)	24	95.2%	50.6%	23.5%	**80.4%**	0.008	0.015	5.26
SmolVLA (expert)	32	67.3%	**67.2%**	35.1%	66.4%	0.002	0.108	5.35
OFT (Llama-2)	32	95.4%	**80.1%**	46.2%	7.3%	0.072	0.342	5.55
GR00T (DiT)	16	83.9%	80.0%	76.2%	**82.6%**	0.010	0.008	5.28

Limitations. α was not swept. We report the published Bhalla et al. [2026] default $\alpha = 1.0$. The conclusion is that at the published hyperparameters T-SAE degrades VLA rollout fidelity on every architecture we tested. Whether a tuned $\alpha < 1$ can encourage temporal smoothness without the latent-code collapse described under Mechanism is an open question.

Training details. We follow the T-SAE recipe of Bhalla et al. [2026] with TopK k=64 in place of its BatchTopK k=20 and without its Matryoshka decoder, so the three arms differ only in pooling and the contrastive term. The total loss adds a symmetric InfoNCE contrastive term to the per-token reconstruction MSE on adjacent latent codes:

$$\mathcal{L} \;=\; \mathcal{L}_{\text{recon}} \;+\; \alpha\,\mathcal{L}_{\text{contr}}, \qquad \mathcal{L}_{\text{contr}} \;=\; -\tfrac{1}{N}\sum_{i} \log \frac{\exp s(z_t^i, z_{t-1}^i)/\tau}{\sum_j \exp s(z_t^i, z_{t-1}^j)/\tau} \;+\; \text{(symmetric)}, \quad (9)$$

with $s(\cdot,\cdot)$ cosine similarity, $\alpha = 1.0$, $\tau = 1.0$. VLA activations are 3-D (T timesteps $\times$ S tokens-per-step $\times$ D). We use `cross_time` adjacency $(t, s) \leftrightarrow (t+1, s)$ as the most direct port of T-SAE's adjacent-token InfoNCE.

All three arms share the activations, the success-only filter, 8$\times$ expansion, Adam (lr $= 10^{-3}$, batch 512, 100 epochs), and per-position normalization. As in the original recipe, in-batch negatives can come from the same episode at a different timestep.

H.3 Dynamic Time Warping Analysis of Cross-Task Trajectories

The *cosine-dominance* override metric used in §4.3 (Eq. 2) compares per-step action-vector directions, which is a local geometric measure. We add a global trajectory-shape measure: Dynamic Time Warping (DTW) on end-effector position trajectories, normalized by warping-path length. For each

Table 13: DTW source-dominance for cross-task injection on $\pi_{0.5}$, alongside per-step cosine source-dominance

Suite	n	DTW→src	DTW→dst	$\text{DTW}_{A,B}$	DTW src-dom. [95% CI]	Cos src-dom.
LIBERO-Goal	270	0.105±0.055	0.232±0.104	0.190	90.7% [86.7, 93.6]	98.9%
LIBERO-Spatial	44	0.121±0.091	0.160±0.089	0.194	86.4% [73.3, 93.6]	97.7%
LIBERO-10 (long)	24	0.338±0.225	0.225±0.073	0.361	45.8% [27.9, 64.9]	95.8%
Pooled	338	0.124	0.222	–	87.0% [83.0, 90.2]	98.5%

cross-task pair (A, B) with prompt B and source A's expert activations injected at layers 16–17, we compute three DTW distances per episode: $d_{\text{src}} = \text{DTW}(\mathbf{x}^{B|A\text{-inj}}, \mathbf{x}^{A})$, $d_{\text{dst}} = \text{DTW}(\mathbf{x}^{B|A\text{-inj}}, \mathbf{x}^{B})$, and the inherent $d_{A,B} = \text{DTW}(\mathbf{x}^{A}, \mathbf{x}^{B})$ as a control. We report the fraction of episodes with $d_{\text{src}} < d_{\text{dst}}$ as DTW source-dominance, paralleling the cosine-dominance override rate.

DTW corroborates the cosine result on `libero_goal` (90.7% source-dominance, n=270) and `libero_spatial` (86.4%, n=44). The 8–12pp gap between cosine and DTW source-dominance on these suites is expected because cosine measures action-vector *direction* step-wise while DTW measures trajectory *shape* globally. An injected episode that briefly aligns with the source then diverges can register as cosine-dominant but DTW-mixed.

The long-horizon `libero_10` suite shows weaker DTW agreement (45.8% source-dominance vs. 95.8% cosine-dominance, n=24). The inherent $d_{A,B}$=0.361 is large and injection-induced trajectory truncation often shortens the rollout, making the truncated trajectory artifactually closer to the shorter destination baseline by raw shape distance. Cosine, normalized per step, is insensitive to this truncation, so the `libero_10` gap reflects trajectory length and leaves the cosine result intact.

Phase-aligned injection. When injecting by step index we assume that the source and destination episodes are in phase. To test this we aligned source to destination steps using dynamic time warping on the end-effector trajectory (9 `libero_goal` pairs × 6 episodes, n=54 per arm, expert L16+L17). We see 94.4% override when using DTW compared to 100% when using step index (Fisher p=0.243). If we shuffle the steps from the source episode we drop to 87.0% (p=0.013), showing that we can detect if order matters. Warping with the same budget (88.9%) and aligning on gripper phase (83.3%) give similar results, with 0.0% as our floor without injection. DTW returns the identity for 5 of the 9 pairs on this checkpoint. On the 3 checkpoints from App. N, where alignment changes about 70% of injected steps, we see 98.1–100% override with step index, 66.7–68.5% with DTW and 70.4–72.2% with shuffling. Our floor is 1.9–5.6%. On X-VLA we see 88.9% override with DTW compared to 100% with step index (p=0.027) and a floor of 0.0% (n=54 per arm). Most of the override is maintained under any reordering of the source steps, and the amount that is lost is the same when aligning on phase and when shuffling.

H.4 $\pi_{0.5}$ Concept-Feature Analysis

Tab. 14 reports which expert layers carry each concept and whether one feature separates the tasks that use it. Later expert layers (L14–L17) carry the strongest signal, and each top feature classifies its tasks at 100% accuracy.

On $\pi_{0.5}$ libero-object, concept-aligned selection (Cohen's $d_f \times \text{freq}_f$) preserves rollout success under ablation (93% → 93%), while action-correlated selection (linear-probe x,y,z,rpy,gripper) drops success by 14 pp because the chosen feature tracks per-step motor output statistics and carries little concept identity.

Causal Necessity of Linear Probe Directions

Linear probes trained to predict action dimensions achieve 97–98% R^2. Projecting out probe directions eliminates linear decodability by construction (R^2 drops to $\approx$ 0). The causal test is behavioral. Rollouts under projection fail (§3.5), indicating the network itself relies on information in these directions. Cross-architecture single-feature ablation sensitivity is reported in Fig. 7 (n=15,096 concept-task pairs across five models).

Table 14: $\pi_{0.5}$ concept features. (a) The three expert layers with the largest summed activation $\sum_t \mathrm{ReLU}(z_{f,t})$ per episode for each concept's top feature, in thousands. (b) Whether the top feature alone separates the LIBERO-10 tasks that use the concept.

(a) Layer specificity

Concept	Best Layer	2nd Best	3rd Best
Action concepts			
PUT	L17 (133k)	L16 (111k)	L15 (86k)
OPEN	L15 (103k)	L16 (79k)	L13 (71k)
PUSH	L17 (412k)	L13 (275k)	L14 (270k)
INTERACT	L17 (274k)	L12 (261k)	L15 (253k)
Object concepts			
BOWL	L16 (114k)	L15 (80k)	L14 (70k)
WINE_BOTTLE	L16 (128k)	L14 (116k)	L17 (104k)
STOVE	L17 (182k)	L15 (164k)	L12 (141k)

(b) Task classification on LIBERO-10

Concept	Tasks Active	Tasks Inactive	Accuracy
PUT	1,3,4,5,6,7,8	0,2,9	100%
OPEN	0,1	2–9	100%
INTERACT	9	0–8	100%

Vision Robustness

Systematic image perturbation across 6,000+ episodes reveals task-dependent visual robustness. We apply horizontal and vertical flips, rotations ($90°$, $180°$, $270°$), center crops (50%, 75%), and object-centric crops. Horizontal and vertical flips break all models tested (0% success), so spatial orientation is rigidly encoded. Rotation and crop robustness varies by task complexity. Simple pick-and-place tolerates mild perturbations while multi-step tasks fail. Object-centric cropping (centering on the manipulation target) outperforms static crops (60% vs. 0–20%), pointing to reliance on manipulation-relevant regions, with full scene context contributing less.

I Ablation Studies

I.1 Temporal Ablation: GR00T Phase-Conditioned Feature Importance

Table 15 shows GR00T N1.5 temporal ablation aggregated over the full 32-layer stack across three LIBERO suites (160 conditions). Early-window ablation (-29.7pp pooled) is nearly as severe as full-episode ablation (-30.5pp), while mid- and late-window ablation cause only -13.8 and -13.6pp. The asymmetry holds on goal and long, and object stays near ceiling in every window. Layer-type-resolved profiles show DiT layers carry the asymmetry (early-window -50pp, similar to full-episode) while Eagle LM layers are flatter (~ -15pp early). The effect scales with task complexity. `libero_long` (303-step mean) shows a -44pp early drop vs. -27pp on `libero_goal` (108-step mean). The per-layer-type decomposition is not separately tabulated here. GR00T is most sensitive during trajectory initiation, after which it tolerates feature ablation.

Table 15: GR00T N1.5 temporal ablation by LIBERO suite (160 conditions across 32 layers).

Suite	Baseline	Full	Early	Mid	Late
LIBERO-Goal	100%	73.1%	73.2%	92.7%	92.4%
LIBERO-Long	100%	53.8%	55.6%	75.3%	75.9%
LIBERO-Object	100%	93.9%	94.0%	95.0%	95.3%
Pooled (160 cond.)	**100%**	**69.5%**	**70.3%**	**86.2%**	**86.4%**

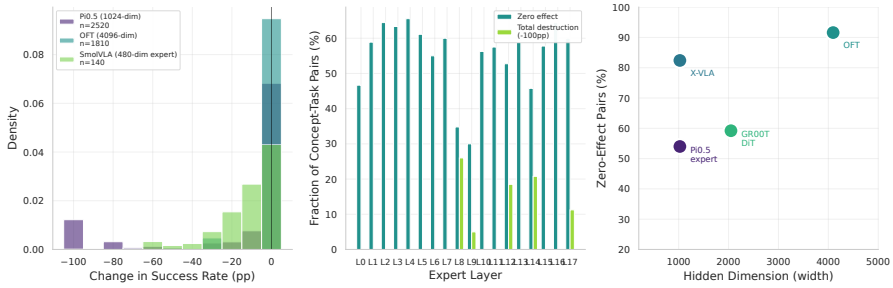
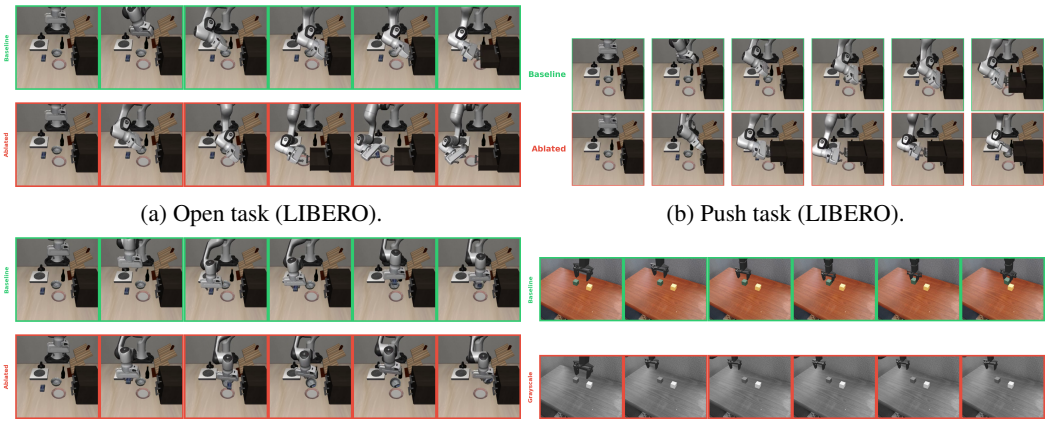


Figure 7: Single-feature ablation sensitivity. Left: change in success per (concept, task) pair for $\pi_{0.5}$ (n=2,520), OFT (1,810) and SmolVLA (140). Middle: per-layer fraction of zero-effect and total-destruction pairs on the $\pi_{0.5}$ expert. Right: zero-effect rate against hidden width for four models.

J Qualitative Results and Additional Figures

On SmolVLA and X-VLA, feature ablation and vision perturbation give binary outcomes: the robot completes the task or fails it, and failed episodes end in a partial reach. For example, SmolVLA 50% center-crop removes button-press spatial context (400-step timeout vs. 67-step baseline), and X-VLA grayscale removes color discriminability for the WidowX stack-cube task (0% success). Figure 8 shows baseline and ablated rollouts. Rollout videos are on Action Atlas (https://action-atlas.com).



(a) Open task (LIBERO).

(b) Push task (LIBERO).

(c) Put task (LIBERO).

(d) Grayscale perturbation (SimplerEnv WidowX).

Figure 8: Baseline (green) and ablated (red) task rollouts. (a)–(c) Feature ablation on LIBERO. (d) Grayscale input on SimplerEnv.

K Per-Architecture Deep Dives

Each subsection gives the architecture's distinctive failure mode and its per-suite table. Baselines match Tab. 2 except where noted, and Fig. 13 summarizes each architecture's best visual-pathway intervention. The ACT control is in App. C.2.

K.1 OpenVLA-OFT

OFT has the highest raw baseline (Tab. 2) and achieves 23/25 on the `libero_10` task-8 seed cluster where $\pi_{0.5}$ scores 0/25. Its main failure mode is structural. If we zero any single layer of the residual stream (0, 8, 16, 24 or 31) we see 0% success on all suites, and if we inject the baseline activations from the visual pathway with a null prompt we see minimal recovery (Tab. 16). OFT has the lowest null-injection recovery of the models tested. With temporal-window null injection on `libero_10`, the nine tasks that succeed at baseline drop to 0% in the early and mid windows. We are able to keep 2 tasks successful when nulling late. Episode-length probes are slightly stronger in the late

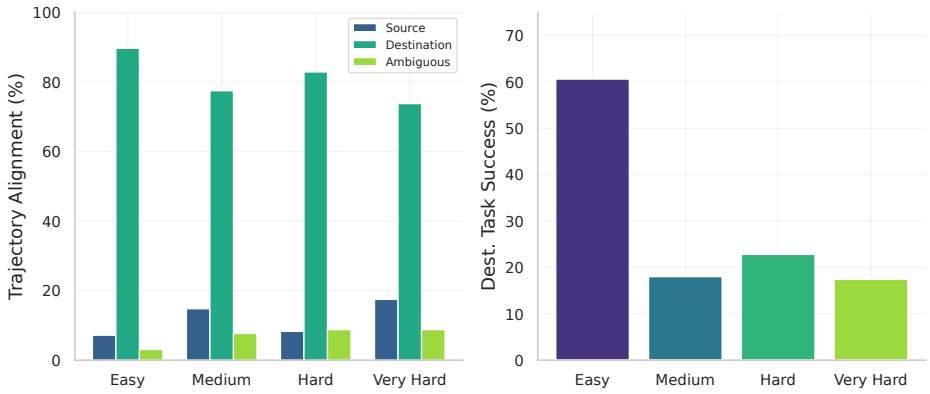


Figure 9: SmolVLA on MetaWorld by difficulty. Left: destination-behavior retention and source-task override under cross-task injection. Right: destination success under injection.

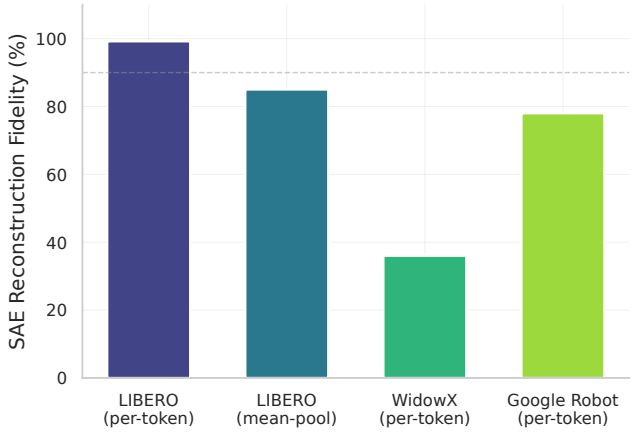


Figure 10: X-VLA SAE rollout fidelity across environments: success with the SAE reconstruction substituted at rollout time, relative to baseline.

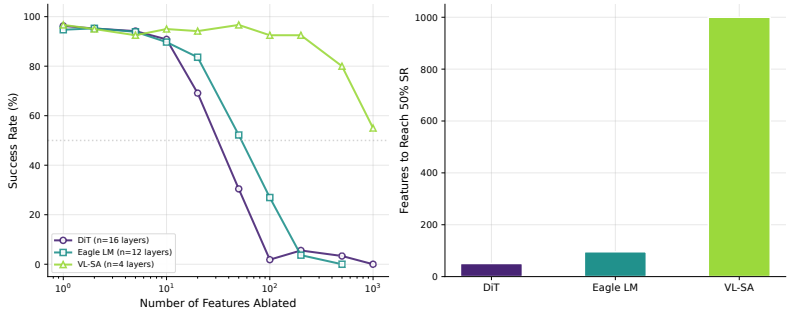


Figure 11: GR00T N1.5 success drop by layer type. DiT layers (16) drop 40–80%, Eagle LM layers (12) drop moderately, and VL-SA layers (4) are the most resilient.

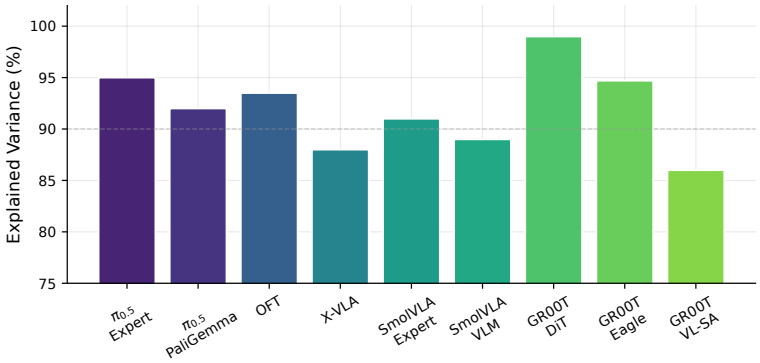


Figure 12: SAE explained variance by architecture and pathway (per-token SAEs, best layer). Dashed line: R^2=0.90.

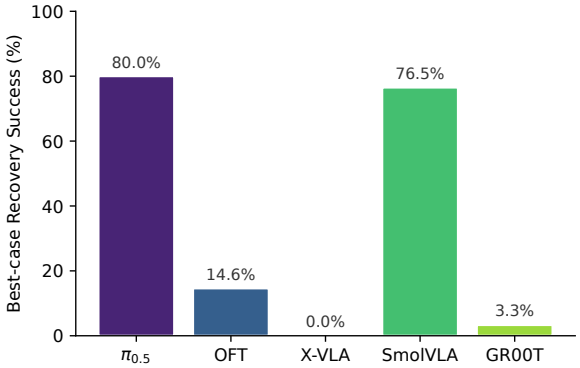


Figure 13: Best-case success under each architecture's visual-pathway intervention, from the rows of Tab. 2: PaliGemma injection under a null prompt ($\pi_{0.5}$), baseline injection under a null prompt (OFT), single-layer zeroing (X-VLA), single expert-layer zeroing on its best suite (SmolVLA), and null-source DiT injection (GR00T).

layers for all suites (early/mid/late R^2: spatial 0.984/0.991/0.986, object 0.719/0.834/0.863, goal 0.859/0.896/0.933, `libero_10` 0.733/0.779/0.827).

Table 16: OFT per-suite deep dive. Zero-layer: any of layers 0/8/16/24/31 nulled under empty prompt (baseline ∼90%). Recovery: null + inject zero-baseline activations. Probe R^2: episode-length probe over 32 layers, n=149/suite.

	Null injection		Same-scene	Probe R^2 (32 L)	
Suite	**Zero-layer**	**Recovery**	**inj. SR**	**Min**	**Mean**
libero_goal	∼0%	33/234 (14%)	87.8% (n=582)	0.845	0.896
libero_object	∼0%	43/294 (15%)	77.8% (n=162)	0.608	0.813
libero_spatial	∼0%	31/216 (14%)	74.0% (n=192)	0.969	0.988
libero_10	0/10 tasks	–	71.8% (n=582)	0.692	0.780

SAE rollout per suite. Tab. 17 breaks the OFT means down by suite. The lower `libero_10` per-token mean (74.5%) is spread across layers (Fig. 14), with no single layer driving it, and all four suites decline after L20.

Per-suite OFT displacement. Table 18 breaks the pooled OFT override of Tab. 3 down by suite, and source-dominant displacement is the majority outcome on every suite. Tie-inclusive override is 77.9% pooled and 73.7–81.9% per suite. Each pair runs in one injection direction, where the $\pi_{0.5}$ analysis runs both. On `libero_object` the destination task still succeeds in 19.7% of injected

Table 17: OFT 3-arm SAE rollout per-suite (32 layers × 25 ep per cell). Per-token wins on every suite by 26–40pp, and mean-pool stays in single digits.

Suite	Per-token	Temporal	Mean-pool
libero_object	**86.6%**	54.6%	9.2%
libero_spatial	**78.2%**	51.8%	6.9%
libero_goal	**81.0%**	43.8%	8.0%
libero_10	**74.5%**	34.5%	5.1%
Mean (4 suites)	**80.1%**	46.2%	7.3%

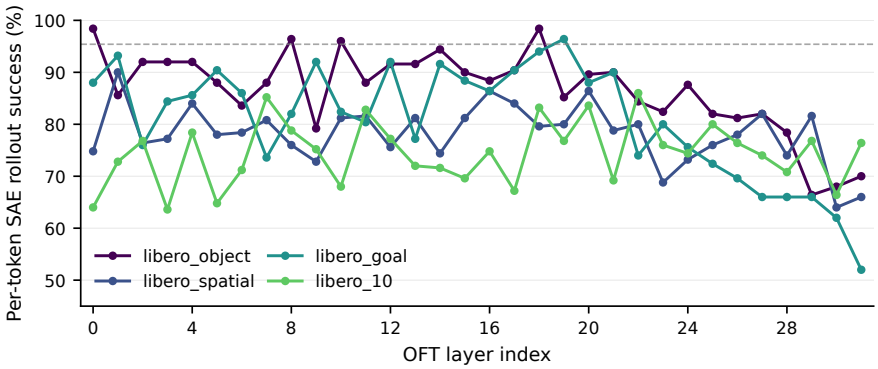


Figure 14: OFT per-token SAE rollout success by decoder layer and LIBERO suite. Dashed line: rollout baseline (95.4%).

episodes, so there the override rate describes the episodes that left the destination task. The OFT cosines are computed from end-effector velocity and may understate displacement where steering acts in action space more than in Cartesian motion.

K.2 Cross-Model Linear Probing

Task-relevant information is linearly decodable on every architecture (Tab. 19). Expert and action pathways encode state dynamics, and VLM pathways encode goal semantics.

K.3 $\pi_{0.5}$

$\pi_{0.5}$ achieves 89.9% on the 1,000-rollout baseline (Tab. 2). The main cause of failure is coordinate-cluster brittleness. For example, on `libero_10` task 8 there is one range of 25 consecutive seeds that gets a 0/25 success rate, while the neighboring ranges get 23/25 and 20/25 on the same task with the same policy. This is a manifestation of the coordinate-bound program from §4.3. With the LIBERO-PRO position perturbations of Zhou et al. [2025], $\pi_{0.5}$ drops from 0.92–0.98 to 0.08–0.38 across all four suites.

Object identity does not predict reach target. Changing the object name while keeping the scene fixed did not change the final end-effector position. One-way ANOVA of the final XYZ position by object name: `libero_object` $F(10, 209)$=1.27, p=0.247, η^2=0.057 (n=220); `libero_goal` $F(9, 170)$=0.30, p=0.973, η^2=0.016 (n=180); `libero_spatial` $F(5, 54)$=2.20, p=0.067, η^2=0.169 (n=60). None of the three is significant at $p<0.05$, and the spatial test is underpowered (n=60). Combined with our results on null prompting, this shows that $\pi_{0.5}$ requires an object name to engage the task but does not use the object name to determine where to go.

Cross-task injection per suite. When injecting into the model we see a decrease in destination success on every suite. `libero_goal` goes from 91.5% down to 0.0% with a cross-prompt and no injection (n=236 for each condition; PaliGemma-all gets 0.4% and expert L16 gets 1.3%). `libero_spatial` (n=34) drops from 100% to 64.7% with a cross-prompt and to 20.6% with its

Table 18: OFT cross-task injection per LIBERO suite, 45 pairs per suite in one direction. Each pair gives 6 episodes, one per injection layer (L0, L8, L16, L24, last, all), and one short `libero_object` episode was dropped. Override is tie-band-conditional (Eq. 2). Dest. success: destination-task success under injection.

LIBERO suite	n pairs	Episodes	Override rate (95% Wilson CI)	Dest success
`libero_goal`	45	270	100.0% (98.3–100.0%)	0.0%
`libero_object`	45	269	97.0% (93.7–98.6%)	19.7%
`libero_spatial`	45	270	90.8% (86.4–93.9%)	0.4%
`libero_10`	45	270	97.7% (94.7–99.0%)	0.0%
Pooled (all suites)	180	1,079	96.4% (94.8–97.5%)	5.0%

Table 19: Cross-model linear probing. Oracle ratio: probe R^2 over oracle R^2, the fraction of ground-truth state linearly decodable from activations. The expert/VLM oracle split is measured on SmolVLA only, and §4.5 gives the $\pi_{0.5}$ and GR00T evidence.

Model	**Probe Target**	**Metric**	**Result**
$\pi_{0.5}$ (expert)	State prediction	R^2	0.45
$\pi_{0.5}$ (PaliGemma)	Prompt classification	Accuracy	99.3%
OFT	Task identification	Accuracy	97.7–100%
OFT	Success prediction	AUC	0.97
OFT	Episode-length (mean)	R^2	0.87
SmolVLA (expert)	Oracle state ratio (h=10)	ratio	0.58
SmolVLA (VLM)	Oracle state ratio (h=10)	ratio	0.13
GR00T (all layers)	Task identification	Accuracy	100%
GR00T (all layers)	Success prediction	Accuracy	96.4%
GR00T (DiT L14)	Success prediction	Accuracy	97.7%

own prompt and PaliGemma-all injection. `libero_10` (n=24) drops from 62.5% to 0% for every injection condition.

K.4 X-VLA

X-VLA reaches 95.2% on the baseline and matches the $\pi_{0.5}$ injection signature (§4.3), including the `libero_10` task-8 seed cluster (0/25, with 24/25 on the neighboring range). Its failures are visual and all-or-nothing, with vision perturbations giving full success or 0% (Fig. 8d). Its null-prompt success depends on the suite, and concept ablation barely moves it (Tab. 20). Its SAEs prefer mean-pooling (80% against 51% per-token over 24 layers), so per-token SAEs lower X-VLA rollout success. The dictionaries are environment-specific (Fig. 10). At layer 0 the LIBERO SAE reconstructs LIBERO activations at normalized MSE 0.010 per-token and 0.006 mean-pool (R^2=0.99), and on WidowX activations it reaches MSE 0.66 (R^2=0.34), falling to $R^2 \leq 0$ by mid-depth.

Table 20: X-VLA per-suite breakdown. Zero any layer: any of 24 tested layers nulled. Null prompt: success under empty string. Zero eff./Δpp: concept ablation zero-effect rate and mean delta. n=4,800 counterfactual episodes and n=2,480 concept-ablation pairs.

Suite	**Baseline**	**Zero any layer**	**Null prompt**	**Zero eff.**	**Δpp**
Goal	98.8%	0%	10%	82.6%	−2.2
Object	98.8%	0%	60%	98.2%	−1.3
Spatial	95.6%	0%	48%	85.2%	−2.6
LIBERO-10	87.6%	0%	28%	74.7%	−2.4

K.5 SmolVLA

SmolVLA reaches 67.3% on the baseline and fails in two ways. Tasks 0 and 4 of `libero_10` sit far below the suite mean of 41.2%. On the task-8 seed cluster it scores 0/25 and recovers only to

12% and 60% on the neighboring ranges. On MetaWorld, injection leaves SmolVLA's destination behavior mostly intact (89.7% easy to 73.8% very hard) and source override low (12.4% aggregate, Fig. 9). Expert concept ablation hurts `libero_spatial` most, while VLM concept ablation hurts `libero_goal` most and helps `libero_object`, consistent with the VLM encoding goal semantics (Tab. 21).

Table 21: SmolVLA per-suite breakdown. Expert zero: success-rate range when zeroing individual expert layers. Zero eff./Δpp: expert-pathway concept ablation (n=1,696). VLM concept ablation (n=210): goal -10.7pp, object $+6.4$pp, spatial -8.5pp, LIBERO-10 $+5.9$pp.

Suite	Baseline	Expert zero	Zero eff. (exp)	Δpp (exp)
Goal	74.8%	60–83%	14.6%	−4.7
Object	87.6%	60–83%	0.0%	−3.9
Spatial	65.6%	47–77%	32.7%	−17.7
LIBERO-10	41.2%	0–33%	20.3%	+0.8

K.6 GR00T N1.5

GR00T achieves 83.9% on the baseline (Tab. 2, spatial*). Zeroing any one of the 16 DiT layers gives a 0% success rate on all suites, while zeroing VL-SA layers keeps 40–80% (Fig. 11), showing that the motor expert is doing most of the work. Similar to X-VLA, language sensitivity depends on the suite. `libero_long` has the lowest concept-ablation zero-effect rate at 42.2% and the highest destruction rate at 11.0% (Tab. 22). The concept-ablation baselines in the table come from a separate batch (n=10 for each task).

Checkpoints. For the goal results we use `Tacoin/GR00T-N1.5-3B-LIBERO-GOAL`, and for `libero_10` we use `liorbenhorin-nv/groot-libero_10-64_40000`, the public checkpoints that had usable baselines (alternatives reach 10% or less). * The spatial export was trained with a batch size of 128 for 20K steps, and the NVIDIA reference reports 97.65% with a batch size of 640. Community fine-tunes range over 68–94%, so we only report the spatial baseline (Tab. 2, †). The qualitative findings hold on the 3 suites that have full coverage.

Table 22: GR00T N1.5 per-suite breakdown. Zero DiT: any of 16 DiT layers nulled. CF "other": counterfactual success under non-baseline prompts (n=180/suite). Zero eff./Δpp: concept ablation across 6,500 pairs.

Suite	Baseline	Zero DiT	CF "other"	Zero eff.	Δpp
Goal	97.0%	0%	18.9%	68.8%	−5.8
Object	99.0%	0%	73.3%	69.6%	−9.9
Long	75.0%	0%	61.7%	42.2%	−5.3

K.7 Operational Summary: Failure-Attribution Rules

These failures give three diagnostics. If the robot confidently goes to the wrong location, it is likely due to corruption in the expert pathway (§4.3). If it does not make a committed motion it is likely due to corruption in the VLM pathway (App. F.1). These apply to models that have separate pathways ($\pi_{0.5}$, SmolVLA, GR00T). A failure on a contiguous range of seeds where the neighboring seeds pass points to brittleness around one layout of object coordinates, rather than a flaw in the whole model. It appears in models where absolute coordinates dominate the action distribution ($\pi_{0.5}$, X-VLA, SmolVLA), and OFT is able to pass on these seeds.

L Dose-Response of Expert Injection

The injection of Tab. 3 replaces the entire expert residual (α=1), so it does not differentiate between a program that switches on at full magnitude and a bias that scales with the magnitude. A switch predicts destination behavior at every partial dose and a step at α=1, and a bias predicts a graded rise as we scale up the magnitude. On $\pi_{0.5}$ `libero_goal` we blend the output of expert layers 16/17

as $(1-\alpha)\,h_{\text{live}} + \alpha\,h_{\text{src}}$, where h_{src} is the source activation from Eq. 1 recorded during one rollout without injection, and we keep the destination prompt fixed. We run 7 pairs of tasks with 5 episodes at each value of $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$. An episode is classified as source or destination when its action-trajectory cosine to the source rollout exceeds or trails its cosine to an uninjected destination rollout by more than τ=0.05 (Eq. 2).

Table 23: Dose-response of expert injection on $\pi_{0.5}$ `libero_goal`, 7 task pairs × 5 episodes per dose. Unanimous: pairs with all 5 episodes source-classified. Cosines are episode means.

α	Source-classified	Unanimous pairs	Dst. success	$\overline{\cos}_{\text{src}}$	$\overline{\cos}_{\text{dst}}$
0.00	0/35 = 0.0%	0/7	34/35 = 97.1%	0.360	0.904
0.25	4/35 = 11.4%	0/7	19/35 = 54.3%	0.475	0.782
0.50	34/35 = 97.1%	6/7	3/35 = 8.6%	0.705	0.312
0.75	35/35 = 100%	7/7	5/35 = 14.3%	0.604	0.154
1.00	35/35 = 100%	7/7	4/35 = 11.4%	0.551	0.147

Episodes that are classified as source go from 0/35 at α=0 to 4/35 at 0.25 and 34/35 at 0.5 before plateauing. Mean cosines also cross in this range (Tab. 23). Since the 0.5 dose alone rules out a step confined to α=1, we read the effect as graded biasing of the expert's trajectory. At low doses we see a decrease in the ability to complete the instructed task (destination success 19/35 for α=0.25) and then redirection at 0.5. Each dose has 35 episodes, which correspond to 7 clusters that share a source rollout. There are no pairs that are unanimously source below 0.5. At 0.5, 6/7 pairs are unanimous, and 7/7 are unanimous at 0.75 and 1.0. The 4 source episodes at 0.25 come from 2 pairs. This is limited to one checkpoint, task suite, and layer pair. Similar to the other simulation runs, we did not match sampling noise (App. O).

M SAE Sparsity Sweep

The pooling comparison of §4.6 uses k=64. We repeat both the per-token and mean-pool rollout on $\pi_{0.5}$ at $k \in \{32, 128\}$ at expert layers 5 and 11, on all four suites of LIBERO (n=30 per cell), using the replacement write-back of Fig. 6. Each `libero_10` arm has its own baseline run.

Table 24: Per-token and mean-pool SAE rollout success on $\pi_{0.5}$ across sparsity k, n=30 per cell, against the uninjected baseline of the same run. Per-token and mean-pool Wilson 95% intervals are disjoint in every cell. Mean-pool uses the replacement write-back.

Suite	Layer	k	Baseline	Per-token	Mean-pool (replacement)
`libero_goal`	L5	32	28/30 = 93.3%	19/30 = 63.3%	0/30 = 0.0%
`libero_goal`	L5	128	28/30 = 93.3%	29/30 = 96.7%	4/30 = 13.3%
`libero_goal`	L11	32	28/30 = 93.3%	15/30 = 50.0%	2/30 = 6.7%
`libero_goal`	L11	128	28/30 = 93.3%	28/30 = 93.3%	3/30 = 10.0%
`libero_object`	L5	32	26/30 = 86.7%	19/30 = 63.3%	0/30 = 0.0%
`libero_object`	L5	128	26/30 = 86.7%	24/30 = 80.0%	1/30 = 3.3%
`libero_object`	L11	32	26/30 = 86.7%	13/30 = 43.3%	0/30 = 0.0%
`libero_object`	L11	128	26/30 = 86.7%	26/30 = 86.7%	0/30 = 0.0%
`libero_spatial`	L5	32	24/30 = 80.0%	16/30 = 53.3%	1/30 = 3.3%
`libero_spatial`	L5	128	24/30 = 80.0%	26/30 = 86.7%	2/30 = 6.7%
`libero_spatial`	L11	32	24/30 = 80.0%	8/30 = 26.7%	0/30 = 0.0%
`libero_spatial`	L11	128	24/30 = 80.0%	28/30 = 93.3%	0/30 = 0.0%
`libero_10`	L5	32	27/30 = 90.0%	11/30 = 36.7%	0/30 = 0.0%
`libero_10`	L5	128	25/30 = 83.3%	23/30 = 76.7%	0/30 = 0.0%
`libero_10`	L11	32	27/30 = 90.0%	8/30 = 26.7%	0/30 = 0.0%
`libero_10`	L11	128	25/30 = 83.3%	21/30 = 70.0%	0/30 = 0.0%

Per-token success is greater than mean-pool in all 16 cells, with a minimum difference of 26.7 pp. When pooling over cells, per-token achieves 314/480 successes (65.4%) and mean-pool 13/480 (2.7%). Increasing k increases per-token success for all suites and layers without changing the order.

Seed and corpus axes. At k=64 on `libero_goal`, three initialization seeds give per-token 80.0% against mean-pool 20.0% at L5, and 86.7% against 6.7% at L11 (n=30 per layer). The

ordering holds for every seed, and the single-seed intervals at n=10 overlap for two of the six seed-layer pairs. Training on 25%, 50%, and 100% of the corpus (n=50 per cell, baseline 96.0%) gives per-token 78.0–92.0% and mean-pool 0.0–26.0%, with disjoint intervals in all six cells.

Scope. The mean-pool column holds for the replacement write-back. By using residual write-back, i.e., adding the SAE edit on top of the original activations, mean-pool success on OFT goes from 0.0% to 73.3%, on GR00T from 2.0% to 82.0% and on X-VLA from 3.3% to 100.0% at single matched cells. There is no result for mean pool with $\pi_{0.5}$ using residual write-back. Per-token will be the same regardless of write-back since there is no pooling involved.

N Fine-Tuning Data Scale

We can also test if coordinate binding is due to there being so little data during task-specific fine-tuning. By decreasing the amount of data we use and performing our injection protocol we can check this. We take $\pi_{0.5}$ (`pi05_libero_base`) and fine-tune it on `libero_goal` with 10%, 30% and 100% of the data using the same recipe (LoRA rank 32, α=64, dropout 0.05, 6,000 steps, batch size 32, learning rate 10^{-4}, warmup 200, seed 42), which translates to 168, 512 and 1,693 episodes. We then perform our 4-condition cross-task injection on expert layers L16/L17 for 9 task pairs $\times$ 6 seeds (n=54/condition). Override is the tie-inclusive rate from Tab. 10, $\Pr[\Delta_t > 0]$.

Table 25: Cross-task injection on $\pi_{0.5}$ as a function of fine-tuning data. Baseline is single-task success over 10 tasks $\times$ 10 episodes. Override, no-injection floor, and shuffled control are n=54 each.

Data fraction	Baseline	Directed override	No injection	Shuffled
10% (168 ep.)	97/100	54/54	1/54 = 1.9%	38/54 = 70.4%
30% (512 ep.)	97/100	54/54	1/54 = 1.9%	45/54 = 83.3%
100% (1,693 ep.)	98/100	54/54	1/54 = 1.9%	40/54 = 74.1%

By lowering the amount of fine-tuning data by 10$\times$ we find that override stays constant at 54/54 and the floor stays at 1/54. Baseline success is also flat and we have all 9 task pairs unanimous at every data fraction. This indicates that we see this phenomenon even at the smallest fraction, which still produces a competent policy. If we use the tie-conditional rule from Eq. 2 (τ=0.05) we get 53/54 when using 10%. The one difference is a single episode within the tie band.

O Physical Replication on a UR10e

This appendix reports a physical campaign that tests the two claims a real rig can falsify directly: that wholesale expert replay redirects which task the policy performs, and that the transplanted program is anchored to workspace coordinates.

Setup. We fully fine-tuned $\pi_{0.5}$ (20k steps) on a Universal Robots UR10e over seven tabletop tasks from 335 teleoperated episodes (60,416 frames at 15 Hz) in the DROID observation format, with two exterior cameras and a wrist camera at 320$\times$180. Fig. 3 uses the exterior camera facing the robot. Baseline success is 76.2% (160/210, 30 trials per task, three observers), from 66.7% to 86.7% per task.

Intervention. The procedure is replicated from simulation. Captured expert activations at L16 and L17 replace the live ones at each inference call (α=1) under the fixed destination prompt.

Flow matching requires matched sampling noise. Wholesale replacement of the expert residual fixes the velocity field, so the denoiser no longer cancels its own initial noise and returns $x_{\text{final}} = a_{\text{source}} + (\epsilon_{\text{dest}} - \epsilon_{\text{source}})$. On the arm this offset showed up as weak override and an over-scaled gripper, so the physical injector reuses the source's per-call initial noise. In simulation, replaying the source's L16/L17 activations with the source's x_T reproduces the source action exactly, and a fresh x_T leaves an error of 2.18 and 3.73 over two draws (action norm 1.03). The simulation runs behind Tabs. 3 and 25 leave this term uncontrolled. Their override rate compares action directions over full episodes and still reaches 54/54, while the arm executes the raw values. Activation-transfer studies on

flow-matching or diffusion heads need to control this term, which discrete and autoregressive heads do not have.

Bidirectional override. Each direction ran directed injection, a matched-scene no-injection floor, and a norm-matched random direction at the same layers and magnitude, with rollouts scored as source, destination, or neither (Tab. 5). Directed intervals exclude both control intervals in both directions, so the effect comes from the content of the injected activations. The 18/26 direction is limited by grasp failure on the compliant plushie. The arm reached for the injected target first in every trial, and the non-source trials diverted only after a failed grasp.

Displaced-object test. Coordinate anchoring predicts that the arm goes to the recorded position of a moved object, vision tracking predicts that it follows the object, and a distribution-shift artifact predicts incoherent motion. In 50 trials, 25 per direction, with the source object moved 10–15 cm, the arm drove to the recorded coordinates every time and closed on empty air at the vacated position. With two blocks swapped into each other's positions it stacks fluently in the wrong color. About a centimeter of displacement already makes the grasp fail. Without injection and with the source prompt, the policy reaches the displaced object, so the coordinate locking belongs to the injected program. After the injected sequence ends, the policy cycles fragments of its trained tasks at their trained coordinates. Outcomes are scored behaviorally, with RGB recorded for every rollout.

P Broader Impact

This work is interpretability research aimed at making VLA decision-making auditable. The artifacts we release (520+ SAE checkpoints, 79 identified concepts, the cross-task-injection rollout corpus, and the feature-exploration platform) are debugging tools and none is a deployment-ready policy. All cross-architecture experiments run in simulation, with one physical replication on a single task pair (App. O).

Positive impact. Mechanistic decomposition supports principled failure analysis. Kill-switch mapping localizes the features and layers a VLA depends on, and spatial-binding analysis flags coordinate-grounded brittleness before deployment. The interventional protocols here (null-injection, cross-task injection, single-feature ablation) narrow a failure to a pathway, a layer, or a feature subspace.

Risks and dual use. Activation injection and feature steering redirect a deployed policy's behavior without retraining, allowing an attacker with white-box access to a VLA's residual stream to reroute a manipulation arm. We mitigate this exposure by releasing interpretability artifacts (SAE checkpoints, rollout videos, and the platform) and no deployable policy or robot control stack. Independently, the coordinate-bound binding documented in §4.3 is itself a deployment hazard. VLAs that pass simulation benchmarks fail under modest scene-position perturbations [Zhou et al., 2025]. Practitioners deploying VLAs for reliable use in real-world environments should run perturbation-style robustness benchmarks [Zhou et al., 2025, Fei et al., 2025] before deployment.

Compute and energy. Total compute and data volume are reported in App. D.7. All training used existing pretrained VLA checkpoints.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The main claims in the abstract and Section 1 (coordinate-bound programs, including the UR10e replication, language sensitivity, pathway specialization, and SAE pooling) is supported by evidence in Sections 4.2–4.7 with sample sizes and confidence intervals.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 4.8 and Appendix B cover benchmark scope (cross-architecture results in simulation, one UR10e replication in Appendix O), counterfactual prompt coverage, cross-task injection confounds, the coordinate-frame control we did not run, and concept identification. Appendix D.4 states which interventions replace the full layer output.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper is empirical and presents no theorems. The SAE objective (Eq. 5), the override metric (Eq. 2), and the temporal-contrastive loss (Eq. 9) are operational definitions, each validated experimentally.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 3, Appendix D, and Appendix H.2 specify SAE architecture (TopK k=64, 4×/8× expansion, tied decoder, AuxK with $\alpha_{\text{aux}} = 1/32$), training (500K samples, batch 4096, 100 epochs, lr 3×10^{-4} cosine), the injection conditions, the counterfactual prompt conditions, the two intervention-hook placements, and the temporal-contrastive matched-ablation protocol. Per-experiment sample sizes are in figure/table captions. All of the code and results are open sourced.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: We provide code, SAE checkpoints, the rollout-validation harness, the feature-exploration platform (Action Atlas, https://action-atlas.com), and the public repository (https://github.com/CWRU-AISM/action-atlas). Reproduction instructions, dependency specs, and per-experiment launchers are included in the supplementary ZIP and codebase.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [Yes]

   Justification: Hyperparameters and choices are reported in Section 3, Appendix D.1, and Appendix H.2. Episode counts per experiment type and per architecture are in Table 7. The intervention-hook protocol distinguishing full-layer-residual, residual-additive, and MLP-only placements is documented in Appendix D.4.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Small-sample and cross-model rates carry 95% Wilson intervals (Tabs. 5 and 10), comparisons use Fisher exact or two-proportion z-tests, and ANOVA effect sizes are η^2.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Compute budget is reported in Appendix D.7: 8×A100-80GB for $\pi_{0.5}$/OFT/large-scale SAE/cross-model work, RTX 5090 (32GB) for GR00T, 2×RTX 4090 (24GB) for SmolVLA/X-VLA, approximately 10,000 A100 GPU-hours in total. Total artifacts ∼7.1 TB (4.3 TB activations, 152 GB SAE checkpoints, 420,000+ rollouts).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics?

Answer: [Yes]

Justification: The work uses public simulation benchmarks (LIBERO, MetaWorld, SimplerEnv, ALOHA), public VLA weights, and a UR10e dataset of 335 episodes teleoperated by the authors. It involves no external human subjects, no personal data, and no scraped private content. Dual-use considerations are addressed in the Broader Impact section.

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Broader Impact section (Appendix P) discusses positive impact (failure-mode debugging tools for VLA operators, brittleness diagnostics), risks (dual-use of activation steering for unintended behavioral redirection), and recommends robustness benchmarking (e.g., LIBERO-PRO) prior to high-stakes deployment.

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse?

Answer: [Yes]

Justification: We release simulation artifacts (SAE checkpoints over LIBERO/MetaWorld/SimplerEnv activations and rollout videos) and the scored UR10e rollout videos and do not release a turn-key real-robot exploit harness. The released SAEs operate on activations of pretrained VLAs; users must obtain VLA weights separately under the original licenses.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and respected?

Answer: [Yes]

Justification: All five VLAs and the ACT control ($\pi_{0.5}$, OpenVLA-OFT, X-VLA, SmolVLA, GR00T N1.5, ACT), all four benchmarks (LIBERO, MetaWorld, SimplerEnv, ALOHA), and the CoTracker3 and Depth Anything 3 models used for Fig. 3 are cited with their original publications. We use checkpoints under their respective public licenses (Apache-2.0 / MIT for the simulation benchmarks; model-specific licenses per the original VLA releases).

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: We release 520+ SAE checkpoints (per-token, mean-pool, and temporal-contrastive variants across five VLAs), the rollout-validation harness for 3-arm pooling comparison, the feature-exploration platform (Action Atlas), the cross-task injection rollout corpus, and the scored UR10e rollout videos. Each is documented in the supplementary materials with directory layouts, filename conventions, and reproduction instructions.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The work involves no human subjects or crowdsourcing.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The work involves no human subjects.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research?

    Answer: [N/A]

    Justification: LLM-based auto-interpretability is an optional tool in the feature-exploration platform for narrating SAE features. The core methods and claims do not use LLMs.